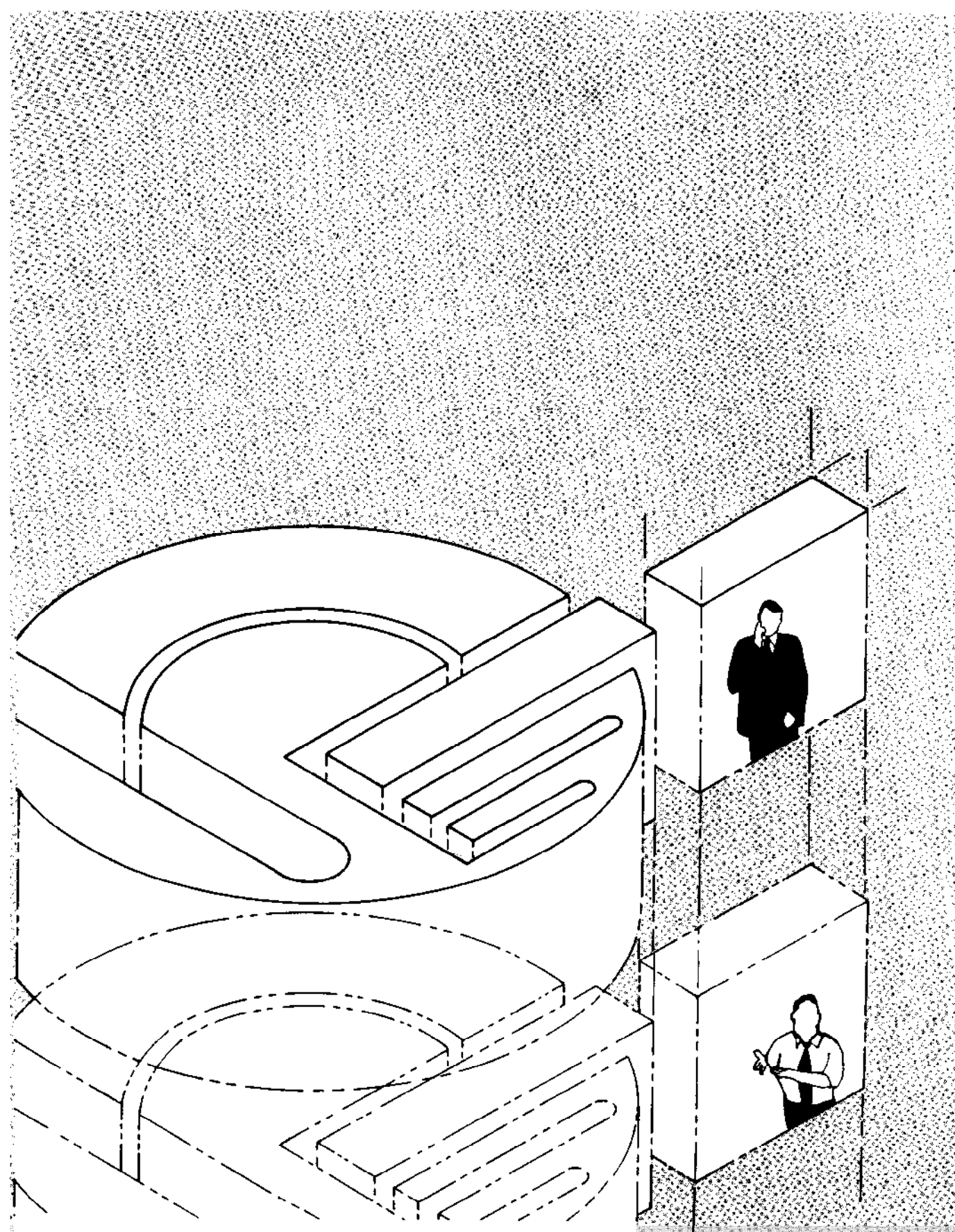


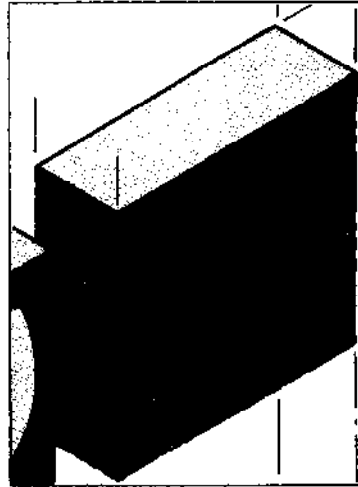


La cuarta parte comprende cuatro capítulos (del 14 al 17), los cuales están diseñados para proporcionar un panorama de la implantación y revisión de los sistemas, la administración de su desarrollo y la selección de sus componentes y servicios de apoyo.

El capítulo 14 trata del aseguramiento de la calidad y presenta criterios para el diseño de sistemas de información confiables y fáciles de mantener. El capítulo 15 se centra en la administración de la implantación del sistema, incluyendo los métodos de entrenamiento para los operarios del sistema y otros usuarios, procedimientos de conversión y revisión

CUARTA PARTE

Implantación, administración del desarrollo, y selección del hardware y software

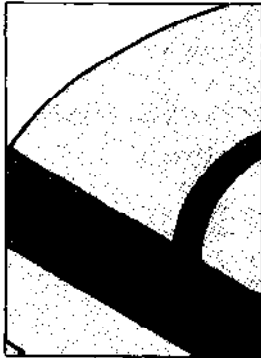


después de la implantación. El capítulo 16 se enfoca a la estimación y coordinación del desarrollo de los sistemas de información. Incluye una discusión de los conceptos de manejo de personal y explica el uso de revisiones y recorridos estructurados para ayudar a alcanzar los estándares de calidad y desempeño de los sistemas de información terminados. El capítulo 17 cubre la adquisición del hardware y software de los sistemas de información, incluyendo las opciones en la elección y contratación de los componentes y el apoyo computacional del sistema, así como los servicios de mantenimiento.

14. Ingeniería de sistemas y aseguramiento de la calidad

GUÍA DE ESTUDIO

Usted comprenderá cómo diseñar y desarrollar software de calidad cuando pueda contestar estas preguntas:



- ¿Qué principios deben guiar el diseño del software?
- ¿Qué quiere decir aseguramiento de la calidad?
- ¿Cómo se prueban los programas de computadora?
- ¿Son lo mismo los errores y las fallas del software?
- ¿Por qué se lleva a cabo el mantenimiento del software? ¿Qué tan frecuentemente se necesita?
- ¿Qué procedimientos de diseño hay que seguir para producir software de calidad?
- ¿Qué papel juegan las gráficas estructurales en el diseño de software?
- ¿Hay más de una forma correcta de diseñar software?
- ¿Qué herramientas se pueden usar para diseñar y documentar software?

OBJETIVOS DEL CAPÍTULO

- Fijar niveles de acoplamiento y cohesión en módulos de software.
- Desarrollar una gráfica estructural particionada.
- Desarrollar un programa de prueba de software.
- Preparar diagramas de flujo estructurado y diagramas de HIPO que describan las actividades del sistema.
- Dibujar diagramas de Warnier/Orr que muestren las relaciones en un proceso.
- Llevar a cabo pruebas de unidades y sistemas para promover el aseguramiento de la calidad.

PALABRAS CLAVE

Acoplamiento
Aseguramiento de la calidad
Biblioteca de pruebas
Caso de prueba
Certificación
Cohesión
Confiabilidad
Diagrama de flujo estructurado
Diagrama de Nassi-Schneiderman
Diagrama de Warnier/Orr
Error
Extensión de control
Falla
Gráfica de estructura

HIPO
Información de control
Parámetros
Prueba alfa
Prueba ascendente
Prueba beta
Prueba de código
Prueba descendente
Prueba de especificaciones
Refinamiento por pasos
Sistema estructurado
Tabla visual de contenido
Validación
Verificación

¿Qué tan probado está lo “probado”?

Terry Young y los demás miembros del equipo de sistemas de información de la compañía estaban en espera de esta reunión con el vicepresidente de mercadotecnia. Como líder del proyecto del sistema de captura de pedidos durante todo el día, Terry pensó que ellos tenían razón en sentirse orgullosos del sistema que recientemente habían terminado.

No sólo habían finalizado el proyecto a tiempo y dentro del presupuesto, sino que éste era el sistema más complejo que el grupo había desarrollado, ligando el manejo de pedidos e inventarios, órdenes de compra, historial de los clientes e incluso control de efectivo y cuentas por cobrar. Un reto adicional era el requisito de que la operación fuera 24 horas al día. El sistema tenía que operar en dos sistemas distintos para permitir un tiempo de mantenimiento del equipo.

Cuando el vicepresidente de mercadotecnia habló con el grupo, sus observaciones subrayaron el éxito del equipo.

“Sabemos que actualmente tenemos al mejor equipo y al mejor personal con el cual una compañía pueda contar. No hay duda de que nuestra habilidad para competir con éxito en el futuro depende de lo que recibimos de nuestra inversión en nuestros sistemas de información. Ustedes amigos, han hecho un excelente trabajo que contribuirá a nuestro provecho y su labor también beneficiará a nuestros clientes, ayudándolos a obtener los mejores productos rápida y fácilmente”.

Terry observó a los demás miembros del equipo de sistemas de información para ver sus reacciones ante los elogios que se les prodigaban. Como ella, ellos parecían estar complacidos con el trabajo realizado. A pesar de haber sido un proyecto tan difícil, ahora se podía ver que bien había valido la pena el esfuerzo.

Terry continuó escuchando al vicepresidente, quien estaba concluyendo sus comentarios:

“Cualquiera puede comprar el mismo hardware que estamos usando, pero nadie más tiene nuestro equipo de trabajo, y nadie cuenta con la confiabilidad que tiene nuestro sistema. Terry Young, líder del equipo de desarrollo, va a hablarles ahora un poco acerca de las amplias pruebas y más pruebas que su gente ha hecho para garantizar la confiabilidad de nuestro sistema. Aunque no soy un experto en control de calidad, estoy seguro de que nuestro sistema no tiene errores. ¡Sin duda alguna, cada instrucción del software ha sido probada hasta la perfección! ¿No es así, Terry?”

La introducción del vicepresidente puso a girar la cabeza de Terry. Al levantarse para presidir la reunión, sus palabras resonaban en su cabeza. ¿No tiene errores?, ¿cada instrucción probada? El vicepresidente parecía no darse cuenta de que no hay tal cosa como un programa depurado o probado en su totalidad, pensó. Bueno, esto sólo tardará un poco más de lo que había planeado, se dijo a sí misma. Miró rápidamente a los otros miembros del equipo de desarrollo y vio que las sonrisas que lucían momentos antes habían cambiado por expresiones tensas.

“Gracias por sus elogios y confianza, señor Steele”, comenzó. “Mis compañeros y yo apreciamos el que usted nos brinde esta oportunidad para hablarle de la confiabilidad del sistema. Comenzaré por decirle lo que hemos hecho para garantizar que el diseño del sistema cumpla las especificaciones y requerimientos para su uso y desempeño deseados. Quisiera explicar por qué **ninguna** prueba de un programa puede garantizar la confiabilidad de un sistema, algo que tal vez lo sorprenda” dijo, mirando directamente al vicepresidente. “Y entonces concluiré diciéndole por qué nuestro equipo cree que este sistema es tan confiable como nos fue posible”.

Terry miró nuevamente a los miembros del equipo y comprobó que sus miradas revivían al dar principio a su presentación.

Mientras más funciones críticas de los negocios, las organizaciones y las actividades cotidianas se están automatizando, hay cada vez más confianza en los sistemas automatizados. Este hecho hace que haya una carga mayor en los analistas de sistemas para garantizar que los sistemas que desarrollan sean adecuados. La calidad de un sistema depende de su diseño, desarrollo, prueba e implantación. Una debilidad en cualquiera de estas áreas pondría seriamente en peligro la calidad y, por lo tanto, el valor del sistema para sus usuarios.

Este capítulo estudia las consideraciones del diseño que conducen a sistemas confiables y bien estructurados. Primero se analizan los objetivos del diseño para la confiabilidad y la facilidad de mantenimiento, continuando con criterios y prácticas de diseño. Se estudian las herramientas específicas que usa el analista para mejorar la confiabilidad y la facilidad de mantenimiento, al igual que los métodos para diseño y documentación.

La última parte del capítulo estudia el aseguramiento de la calidad y la prueba de sistemas y software. Veremos como las filosofías, estrategias y técnicas se combinan para producir el software confiable que desean tanto el analista como el usuario.

Comenzaremos el estudio de la ingeniería de sistemas, explorando los objetivos generales del diseño de software.

OBJETIVOS DE DISEÑO

Los dos objetivos operacionales de diseño que siempre buscan las personas que los desarrollan son la confiabilidad y la facilidad de mantenimiento del sistema. Esta sección centra su atención en la importancia de estos objetivos y los medios para alcanzarlos.

Diseño de sistemas confiables

Se dice que un sistema tiene *confiabilidad* si no produce fallas costosas o peligrosas al usarse de manera razonable, es decir, de tal forma que un usuario típico espera que sea normal. Esta definición reconoce que los sistemas no siempre se utilizan en la manera en que los diseñadores lo esperan. Existen cambios en las formas en que los usuarios usan el sistema y también en las operaciones de la empresa. Sin embargo, hay ciertos pasos que los analistas deben dar para garantizar que el sistema sea confiable cuando se lo instala y que la confiabilidad se puede mantener después de la implantación.

Enfoques de la confiabilidad

Hay dos niveles de confiabilidad. El primero es en el que el sistema cumpla con los requerimientos correctos. Por ejemplo, se espera que un sistema tenga características o controles específicos de seguridad, contruidos dentro de él a petición de los usuarios. Pero si el diseño no los especifica y permite la pérdida de fondos o mercancía durante mucho tiempo antes de que alguien detecte el problema, el sistema no es confiable. La confiabilidad a nivel diseño es posible sólo si el analista lleva a cabo una determinación cabal y efectiva de los requerimientos del sistema. Se necesita un estudio cuidadoso y completo del sistema para satisfacer este aspecto de la confiabilidad. Ya hemos visto varios aspectos del estudio de los sistemas.

El segundo nivel de la confiabilidad del sistema tiene que ver con los resultados reales que el sistema entrega al usuario. En este nivel, la confiabilidad del sistema se entrelaza con la ingeniería del software y su desarrollo.

Un *error* aparece cuando el sistema no produce los resultados esperados. Aunque es cierto que ningún sistema se depura o prueba en su totalidad, ni se puede demostrar que sea correcto —un hecho que alarma a muchos usuarios y aspirantes a programador— los errores no se limitan solamente al uso correcto de la sintaxis de programación.

La industria de la computación, en gran medida por medio del trabajo de Glenford Myers, ha distinguido entre los errores y las fallas. Una *falla* es la aparición de un error del software, con un cierto peso dado por su seriedad. Por ejemplo, si se desarrolla un programa sobre inventarios que trunca en vez de redondear los precios al calcular el valor de las mercancías hay un error si las especificaciones señalan redondeo. Pero esto puede no ser de consecuencia para el

Tabla 14.1 Enfoques de la confiabilidad

ENFOQUE	DESCRIPCIÓN	EJEMPLO
Prevención de errores	Previene la ocurrencia de errores en el software.	Es imposible en sistemas grandes.
Detección y corrección de errores	Reconoce los errores cuando se les halla y corrige el error o el efecto del error de tal forma que el sistema no falla.	Atrapa y modifica pasos aritméticos ilegales; compensa los valores inesperados de los datos.
Tolerancia a errores	Reconoce los errores cuando aparecen, pero permite que el sistema siga trabajando con una ejecución degradada o aplicando reglas que instruyen al sistema para que continúe el proceso.	Inhabilita parte del sistema. No lleva a cabo algún proceso pero mantiene operando al sistema.

usuario, que de hecho no lo reconoce como una falla. Sin embargo, si el programa se brinca ciertos artículos con regularidad o indica que no hay en existencia cuando en realidad sí lo están, hay una falla seria.

Prevención de errores Hay tres enfoques para la confiabilidad (tabla 14.1). Con la prevención de errores, los desarrolladores y programadores hacen todos los intentos por evitar que los errores ocurran. El propósito de los métodos y técnicas estructurados que se estudian en el presente capítulo es cumplir con este objetivo. Otra forma de buscar dicho objetivo es poner énfasis en la identificación temprana y cuidadosa de los requerimientos del usuario.

Los analistas deben considerar que es imposible alcanzar por completo este objetivo. Los errores pueden ocurrir no obstante los mejores esfuerzos de gente muy competente.

Detección y corrección de errores Este método usa características del diseño que detectan errores y hacen los cambios necesarios para corregir ya sea el error, mientras el programa esté en uso, o el efecto sobre el usuario, de tal forma que no ocurra una falla. En el capítulo 9 se estudió la corrección de los errores del usuario, tal como un mal deletreo de los términos importantes o la introducción de comandos inválidos. La detección de errores en un programa se hace de manera similar. Por ejemplo, un programa que calcule la productividad de un mesero o mesera en un restaurant, dividiendo la renta total de las comidas servidas entre el número de horas trabajadas, no debería fallar si los empleados no sirven nada. Si se avecina una fuerte nevada y los clientes se abstienen de ir al restaurant, los empleados acumularán tiempo de trabajo pero no tendrán ventas. El programa debería detectar el error de dividir entre cero y corregirlo para que el sistema

se mantenga trabajando en forma adecuada. Desafortunadamente, muchos programas fallan cuando ocurre una situación como ésta. Aun cuando esto pueda no ocurrir durante varios años después de instalar el sistema, el error está allí desde el día en que se desarrolló el sistema. La falla ocurre posteriormente.

Tolerancia de errores Las estrategias de tolerancia de errores mantienen al programa en ejecución aun en la presencia de errores. La Administración Nacional para la Aeronáutica y el Espacio de los E.E.U.U. (NASA), por ejemplo, diseña sus sistemas para que toleren errores mediante el uso de hardware redundante. En un programa espacial, se usan computadoras redundantes a bordo y el voto de las computadoras se utiliza para procesar datos en paralelo, de tal forma que se puedan comparar los resultados. Dos computadoras procesan los datos de localización, corrección de curso y uso del combustible y compara los resultados con los de las otras dos computadoras que procesan los mismos datos. Se dispone de una quinta computadora para romper un empate en caso de que éste ocurra. En caso necesario, una sexta computadora guardada en un compartimiento de almacenaje accesible puede reemplazar rápidamente a cualquiera de las otras si se ha dañado o fallado.

Otra forma de la tolerancia de errores es el uso del procesamiento degradado. Con esta estrategia, el usuario recibe menos servicio del que el sistema está diseñado para proporcionar, pero esto se considera una mejor alternativa a los casos en que no dé servicio del todo. Por ejemplo, muchas de las instalaciones de generación y distribución de energía eléctrica en América del Norte se controlan mediante computadora. Suponga que en un día extremadamente caluroso, el sistema se sobrecarga y la computadora del centro de control es incapaz de procesar adecuadamente la distribución de cargas para cubrir la demanda de energía. En vez de arriesgarse a dañar la red de distribución de energía, la computadora automáticamente desconecta parte de la red. Al proporcionar un servicio degradado, la computadora tolera un error del software sin fallar.

Causas de los errores

Los aspectos de software en el diseño de sistemas son distintos de los considerados sobre la confiabilidad del hardware. En el hardware, por ejemplo, todos los errores de diseño se reproducen en cada copia del artículo fabricado. Sin embargo, los sistemas de aplicación son a menudo únicos y los errores de diseño no se distribuyen ampliamente. Por supuesto, si usted está trabajando en un sistema que se venderá comercialmente, existe un considerable interés en desarrollar y comercializar un paquete de software que no tenga errores de diseño.

Los defectos o errores de fabricación se introducen durante el proceso de producción. No son una propiedad del diseño y, de hecho, es posible que no aparezcan en cada artículo producido. Los errores

de fabricación pueden aparecer en artículos producidos durante un periodo de tiempo específico, ya sea por problemas desconocidos con la calidad del material o por personas recientemente asignadas a un paso del proceso. En los sistemas de software, el equivalente de los errores de fabricación es la pequeña probabilidad de que, al hacer las copias en disco o cinta del programa para su distribución, se introduzcan ciertos errores. Sin embargo, este problema ocurre rara vez y no debe ser de mayor interés para el analista.

Aparecen fallas en el hardware cuando el equipo se usa y empieza a deteriorarse. No hay un equivalente en el software; es decir, no puede ocurrir que el software se vuelva inútil debido a que se deteriora. El medio en que se transporta (como cintas magnéticas o discos) se puede deteriorar o dañar, pero el software no.

Por lo tanto, el principal problema del software es el de diseñar y desarrollar software que no falle. Conviene recordar que, como lo señaló el líder del proyecto en el ejemplo del principio del capítulo, es imposible demostrar que no existen errores en un sistema particular.

Las causas de error que interesan al analista son 1) no obtener los requerimientos correctos, 2) no entender correctamente los requerimientos y 3) no traducir los requerimientos de una forma clara y entendible, de manera que los programadores los implanten correctamente.

Los capítulos anteriores acerca del análisis de sistemas describen los métodos para conducir un estudio efectivo del sistema e identificar los requerimientos correctos. Las técnicas de diseño y los criterios de diseño para las salidas, entradas, archivos y procesos garantizarán, si se aplican adecuadamente, que los requerimientos sean los correctos.

La transición del diseño del sistema al desarrollo del software es una oportunidad adicional para introducir errores de traducción. Estos son el resultado de que el programador no comprende o no interpreta bien las especificaciones de diseño producidas por el analista. Recíprocamente, también aparecen cuando los analistas fuerzan a los programadores a traducir especificaciones incompletas. En el último caso, el programador se ve obligado a hacer decisiones de diseño mientras codifica el software.

Cuando ocurren tales malentendidos y se hace la implantación antes de detectarlos, el resultado es la necesidad de mantenimiento. Por supuesto que puede haber otras razones para el mantenimiento de sistemas, como lo estudia la siguiente sección.

Diseño de sistemas fáciles de mantener

Cuando se instalan los sistemas, generalmente se usan por periodos largos. La vida promedio de un sistema es de 4 a 6 años y las aplicaciones más antiguas a menudo están en uso durante más de 10 años. Sin embargo, este tiempo de uso trae consigo la necesidad de mantener continuamente el sistema. Debido al uso de un sistema después de

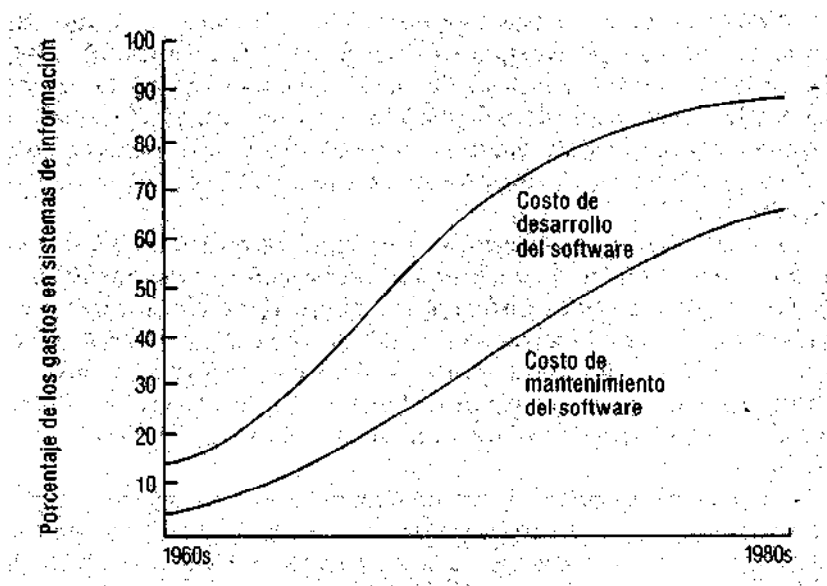


FIGURA 14.1
Tendencias en los costos de software y mantenimiento.

que se ha implantado por completo, los analistas deben tomar sus precauciones para garantizar que la *necesidad* del mantenimiento se controle mediante el diseño y la prueba y la *capacidad* para llevarlo a cabo se asegure mediante adecuadas prácticas de diseño.

Puntos del mantenimiento

Se han realizado muchos estudios en empresas privadas, en universidades y en el gobierno para conocer los requerimientos de mantenimiento de los sistemas de información. De los estudios realizados generalmente han concluido lo siguiente:

1. Del 60 al 90% del costo total del software durante la vida de un sistema se gasta en mantenimiento (Fig. 14.1).
2. A menudo, el mantenimiento no se hace de manera muy eficiente. En algunos casos documentados, el costo del mantenimiento, medido sobre la base del costo de escribir cada instrucción, es más de 50 veces el costo inicial de desarrollo del sistema (Boehm, 1976).
3. La demanda de software está creciendo con más rapidez que la oferta. Muchos programadores están trabajando más tiempo en el mantenimiento de sistemas que en nuevos desarrollos. Los estudios han mostrado que en algunas instalaciones, dos terceras partes de los programadores dedican su tiempo al mantenimiento del software. Existe un rezago en el trabajo de nuevos desarrollos. Además, hay un atraso oculto; aquellos trabajos que los usuarios ni siquiera se preocupan por pedir ya que saben que tardará años antes de que el desarrollo pueda comenzar.

Varios estudios en mantenimiento han examinado los tipos de tareas llevadas a cabo en el mantenimiento (Lientz y Swanson, 1980). La tabla 14.2 resume las clases de mantenimiento encontradas en los ambientes de sistemas de información. Una vez que los sistemas se instalan, la necesidad de depurar y corregir errores o las fallas en caso de emergencia es relativamente baja: menos del 20% de las tareas son de corrección.

Los sistemas de información y las organizaciones a las que sirven están en un estado de flujo constante. Por lo tanto, el mantenimiento de los sistemas también implica adaptaciones de versiones anteriores del software. Aproximadamente, el 20% de todo el mantenimiento se lleva a cabo para adecuarse a los cambios en los reportes, archivos y datos. Esto también incluye las adaptaciones necesarias cuando se instala nuevo hardware o software en un centro de procesamiento particular.

La mayor cantidad de trabajo de mantenimiento es para cumplir con peticiones de los usuarios, mejorar la documentación o recodificar los componentes del sistema para una mayor eficiencia. El 60% de todo el mantenimiento es con este fin. Aun así, muchas de las tareas en esta categoría se pueden evitar si la ingeniería de sistemas se lleva a cabo en forma adecuada.

Diseños fáciles de mantener

Las claves para reducir la necesidad de mantenimiento, al igual que para hacer posible que se realicen las tareas esenciales más eficientemente, son:

1. Definir con mayor precisión los requerimientos del usuario durante el desarrollo del sistema.
2. Preparar lo mejor posible la documentación del sistema.
3. Usar métodos más efectivos para el diseño de la lógica del procedimiento y comunicárselos a los miembros del equipo del proyecto.
4. Hacer un mejor uso de las herramientas y técnicas existentes.
5. Dirigir el proceso de ingeniería de sistemas en forma efectiva.

Como lo indican los comentarios anteriores, el diseño es tanto un proceso como un producto. Las prácticas de diseño que se siguen para el software afectan en forma dramática la facilidad de mantenimiento de un sistema: las prácticas de diseño adecuadas dan por resultado un producto al cual puede dársele mantenimiento.

GRÁFICAS DE ESTRUCTURA DE PROGRAMAS

Los diseños bien estructurados facilitan el mantenimiento de un sistema. Un *sistema estructurado* es modular y desarrollado en forma

CATEGORÍA	ACTIVIDAD	FRECUENCIA RELATIVA
Correctivo	Ajustes de emergencia, depuración rutinaria	20%
Adaptivo	Inclusión de cambios a los datos y archivos, así como al hardware y software del sistema	20%
Perfectivo	Mejoras solicitadas por los usuarios, mejoras en la documentación, recodificación para mejorar la eficiencia computacional	60%

descendente, es decir, separado en componentes manejables. Los módulos deben diseñarse de forma que tengan un mínimo efecto sobre los demás módulos del sistema. Las conexiones entre módulos son limitadas y la interacción de datos es mínima. Se pretenden tales objetivos de diseño para que mejore la calidad del sistema y que faciliten las tareas de mantenimiento.

Los diagramas de estructura estudiados en el capítulo 4 muestran la relación de los módulos de procesamiento en el software. Esta sección estudia el desarrollo y uso de los diagramas de estructura.

Propósito de los diagramas de estructura

Un *diagrama de estructura* es una herramienta de diseño que muestra gráficamente las relaciones entre los módulos de un programa. Presenta cuáles módulos interactúan dentro de un sistema y también muestra gráficamente los datos que se comunican entre varios módulos.

Los diagramas de estructura se desarrollan *antes* de escribir el código del programa. No se pretende que expresen la lógica del procedimiento, tarea que se deja a los diagramas de flujo y pseudocódigo (una forma de español estructurado que se usa para describir la lógica del programa). Tampoco delinear la interfase física real entre las funciones de procesamiento (un objetivo de HIPO, como veremos en un momento). En vez de eso, indican las transferencias de datos entre módulos del sistema.

Simbología

Por conveniencia y facilidad de comunicación entre los desarrolladores de sistemas, se usa una simbología común en la elaboración de diagramas de estructura. Los módulos del programa se identifican mediante rectángulos (Fig. 14.2), con el nombre del módulo escrito

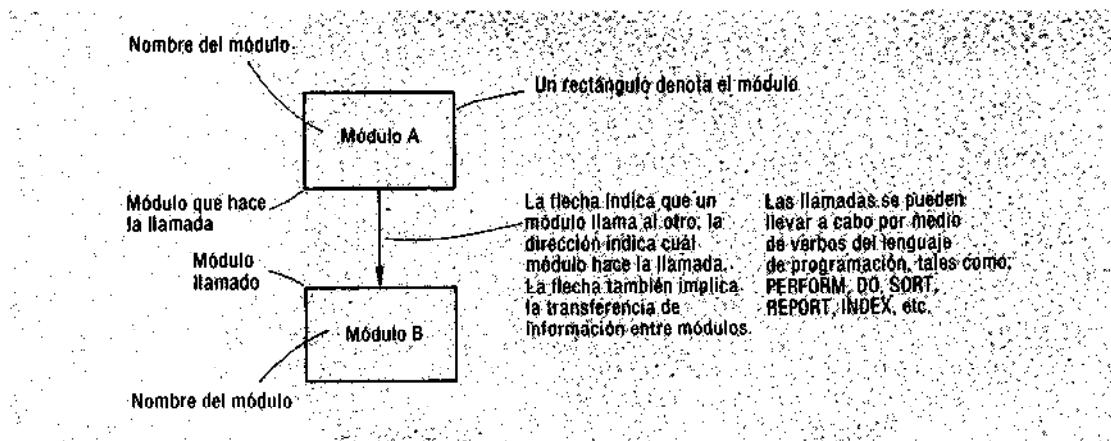


FIGURA 14.2
Notación usada en
diagramas de
estructura.

dentro del rectángulo. Las flechas indican las llamadas, las cuales son cualquier mecanismo para invocar un módulo particular. En los lenguajes de programación tradicionales, las llamadas se hacen ejecutando enunciados PERFORM y DO. En los lenguajes de cuarta generación y los sistemas de base de datos, las llamadas pueden ser macro instrucciones o comandos de alto nivel (tales como PREPARE, REPORT o INDEX). Para los fines del diseño, el mecanismo no es tan importante como reconocer que existe una relación entre los módulos individuales.

Las notas en el diagrama de estructura indican los parámetros que se transfieren y la dirección de movimiento de los datos. En la figura 14.3, vemos que los módulos A y B interactúan. Los datos identificados como X y Y se pasan al módulo B, el cual a su vez regresa el valor Z.

Un módulo que hace una llamada puede interactuar con más de un módulo subordinado. La figura 14.3 también muestra al módulo L, llamando a los módulos subordinados M y N. M es llamado con base en un punto de decisión en L (lo cual se indica con un pequeño rombo), mientras que N es llamado con base en un ciclo iterativo de procesamiento (denotado por el arco al principio de la flecha que hace el llamado).

Transferencia de datos

Cuando un módulo llama a otro, el módulo que hace la llamada puede enviar datos al módulo llamado de tal forma que pueda llevar a cabo la función descrita en su nombre. De la misma forma, el módulo llamado puede producir datos que se transfieran de nuevo al módulo que hizo la llamada.

Los parámetros son datos necesarios en el módulo llamado para realizar el trabajo necesario. Se usa una flecha pequeña con un círculo

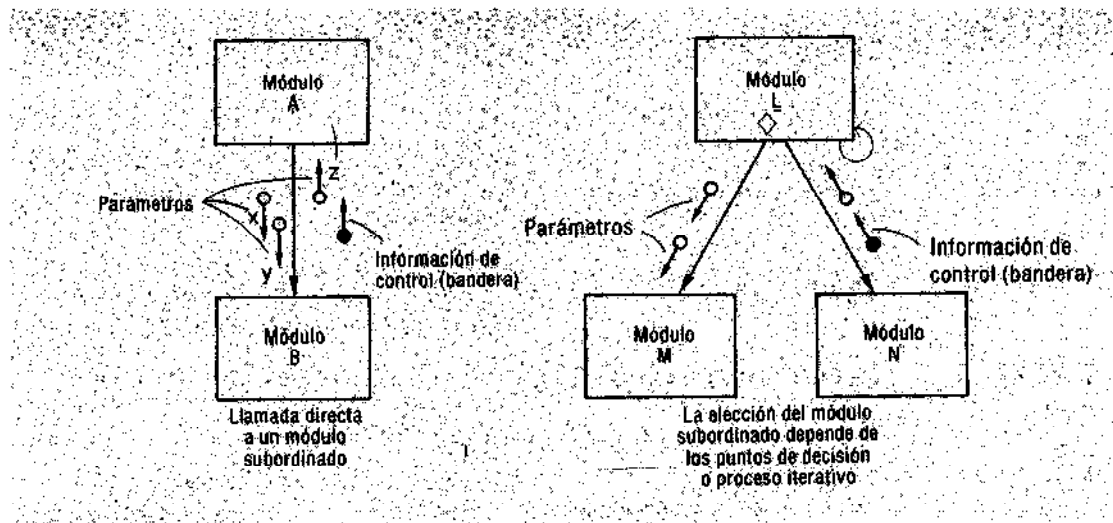


FIGURA 14.3
Anotaciones y transferencia de datos en diagramas de estructura.

hueco en su base para denotar la transferencia de parámetros. También se transfiere la *información de control* (a veces llamada *bandera*). Su propósito es ayudar a controlar el proceso indicando la ocurrencia de, digamos, errores o condiciones de fin de archivo. Una flecha pequeña con un círculo lleno identifica la información de control. Una anotación breve describe el tipo de información transferida (Fig. 14.3).

Los diagramas de estructura son una herramienta para ayudar al analista a desarrollar software que cumpla los objetivos del buen diseño de software. Estudiaremos estos objetivos en la sección siguiente.

DISEÑO DE SOFTWARE

Los siguientes principios deben guiar el diseño del software (Tabla 14.3):

- *Modularidad y fragmentación*
Cada sistema debe estar formado por una jerarquía de módulos. Generalmente, los módulos de niveles inferiores son menores en alcance y tamaño comparados con los módulos de nivel superior y sirven para fragmentar procesos en funciones separadas.
- *Acoplamiento*
Los módulos de un sistema deben tener poca dependencia entre sí.
- *Cohesión*
Los módulos deben llevar a cabo sólo una función de procesamiento.

PRINCIPIO	DESCRIPCIÓN	OBJETIVOS
Modularidad y fragmentación	Diseño de un sistema como una jerarquía de módulos	Diseñar la estructura en forma descendente con módulos que realicen funciones específicas.
Acoplamiento	La fuerza de las relaciones <i>entre</i> módulos	Maximizar la independencia entre los módulos <i>minimizando el acoplamiento</i> (reducir el acoplamiento del módulo).
Cohesión (integración)	La fuerza de las relaciones <i>dentro</i> de un módulo	<i>Maximizar la cohesión</i> ; los elementos altamente relacionados deben estar en el mismo módulo.
Extensión de control	Número de módulos subordinados al módulo que hace la llamada	Limitar la extensión de control de 5 a 7 módulos.
Tamaño	Número de instrucciones que componen a un módulo	Limitar el tamaño de forma que la función de todo el módulo se centre en un sólo propósito.
Uso compartido	Uso de un módulo por otros módulos	Evitar la duplicación permitiendo que los módulos sean llamados por otros que necesitan la función de cada uno.

- *Extensión de control*
Los módulos deben interactuar con y coordinar las funciones de un número limitado de módulos de nivel inferior.
- *Tamaño*
El número de instrucciones contenidas en un módulo debe ser limitado; el tamaño del módulo es generalmente pequeño.
- *Uso compartido*
Las funciones no deben repetirse en módulos separados, sino establecerse en un único módulo que se pueda utilizar por cualquier otro cuando sea necesario.

Cada uno de estos principios se examinará con más detalle, para demostrar con ejemplos como se aplican.

Estructura descendente de módulos

Los métodos descendentes se usan en todo el proceso de análisis y diseño. El valor de usar un enfoque descendente, empezando en los niveles generales para comprender el sistema y moverse en forma gradual hacia los niveles de mayor detalle, se estudió junto con la etapa de análisis. En el proceso de moverse de arriba hacia abajo, cada

componente se “hizo estallar” en más detalles. Un diagrama de flujo de datos se convierte en varios en el siguiente nivel inferior (Fig. 14.4).

Durante el estudio del diseño de entrada y menú, se enfatizó el enfoque descendente. El menú principal contiene varias opciones. Al escoger una de ellas se obtiene otro menú en el cual se presentan al usuario opciones más detalladas. Esta estructura proporciona a los usuarios un método fácil de entender para usar el sistema y elegir las opciones. No tienen que hacer todas las decisiones al mismo tiempo sino solo una a la vez.

El método descendente también se usa ampliamente en la ingeniería de sistemas y diseño de software. Cada función que el sistema lleva a cabo se identifica primero y después se desarrolla con mayor detalle. Los diseñadores de programas llaman a esto *refinamiento por pasos*: los procedimientos y procesos se desarrollan uno a la vez, desde lo general hasta lo particular.

Por ejemplo, un sistema de contabilidad está formado por varios módulos separados que se llaman uno a la vez, cuando los usuarios indican la función particular que desean llevar a cabo (Fig. 14.5). A su vez, el módulo de nivel superior llama a uno o varios módulos de nivel inferior hasta que la función deseada se realice.

Acoplamiento

El acoplamiento, explicado aquí, y la cohesión, examinada en la siguiente sección, miden aspectos opuestos del diseño de software (Fig. 14.6). El *acoplamiento* se refiere a la *fuerza de la relación entre módulos* de un sistema. En general, los buenos diseñadores buscan desarrollar la estructura de un sistema de tal forma que un módulo tenga poca dependencia de cualquier otro módulo.

Un acoplamiento holgado minimiza la interdependencia entre los módulos. Se puede alcanzar esto mediante las siguientes formas:

- Controlar el número de parámetros que se transfieren entre los módulos.
- Evitar la transferencia innecesaria de datos a los módulos que se llamen.
- Transferir datos (ya sea hacia arriba o hacia abajo) sólo cuando sea necesario.
- Mantener las relaciones superior/inferior entre los módulos que llaman y los que son llamados.
- Transferir datos, no información de control.

Conviene considerar la forma en que se transfieren los datos en un sistema de contabilidad. Al editar un registro de un proveedor (para el módulo de cuentas por pagar), la figura 14.7 muestra dos diseños posibles para editar el registro. En el primero, tipificado por un fuerte acoplamiento, el cual no es deseable, el módulo que hace la llamada

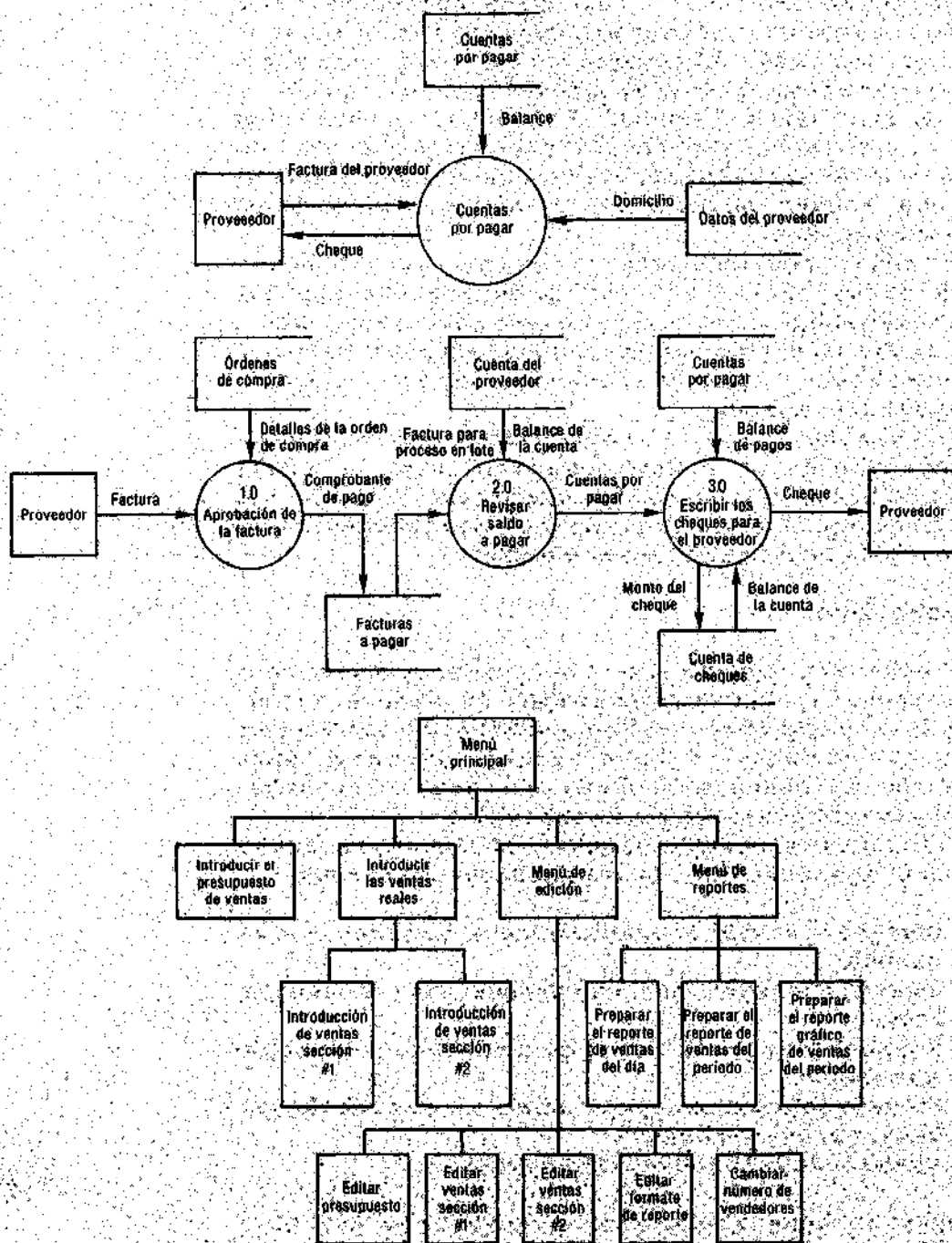


FIGURA 14.4
Métodos descendentes.

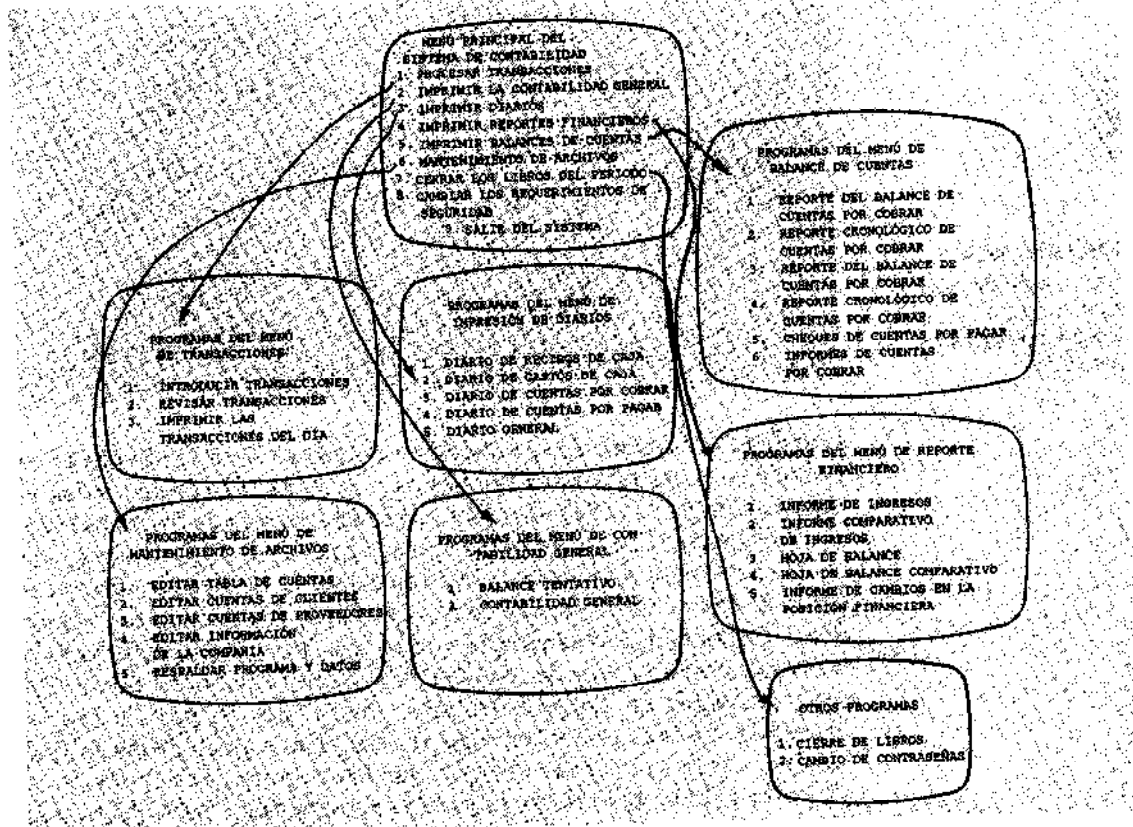


FIGURA 14.5

Programas para operar un extenso sistema de contabilidad modularizado.

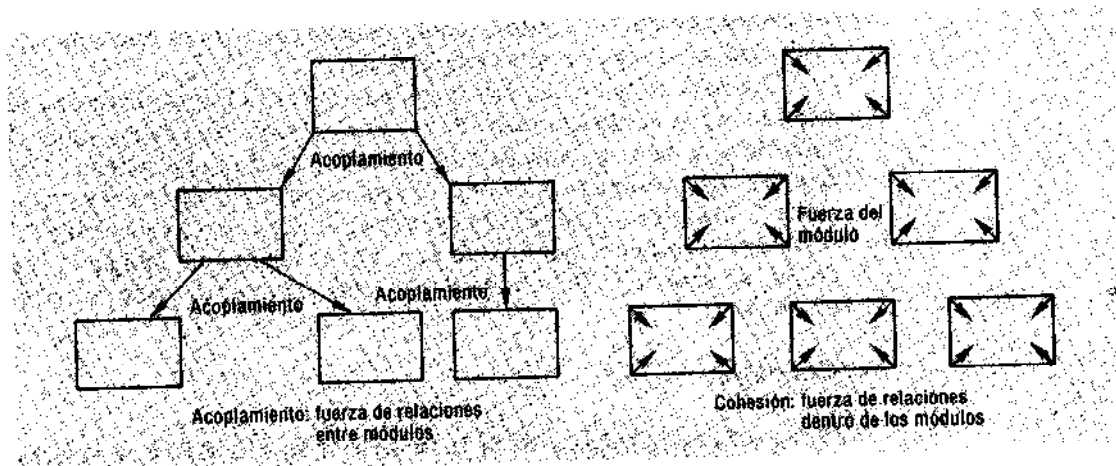
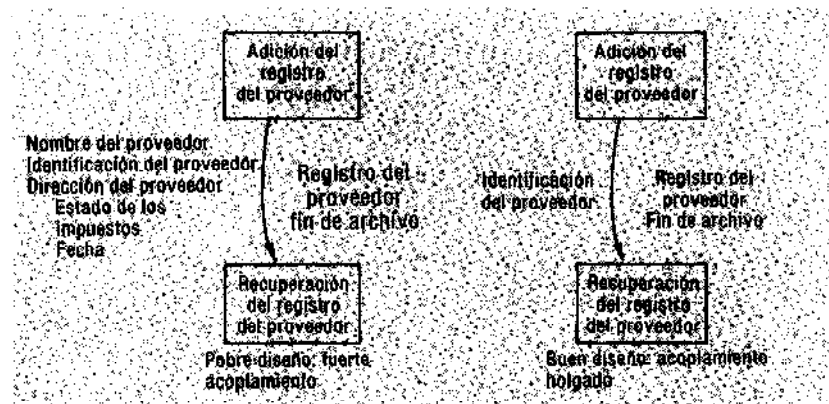


FIGURA 14.6

Acoplamiento y cohesión en el diseño de software.

FIGURA 14.7
Acoplamiento y fuerza de las relaciones entre módulos



transfiere el nombre del proveedor, su número de identificación, su domicilio, el estado de sus impuestos y la fecha. El módulo llamado regresa el registro del cliente, junto con una bandera de fin de archivo.

Compare esto con la versión con acoplamiento holgado en la cual sólo se transfiere el número de identificación del proveedor para recuperar la misma información. Pero este diseño no sólo mueve menos datos (solamente los necesarios), sino que hay menos dependencia entre los módulos. Sólo se requiere de la identificación del proveedor para distinguir un registro del otro. Puesto que es posible que la identificación sea la llave del registro, también es poco probable que cambie. Otros campos del registro pueden cambiar. Por lo tanto, la alternativa de acoplamiento holgado es más adecuada para alcanzar el diseño deseado y los objetivos de mantenimiento.

Evite varias características de un diseño pobre (Fig. 14.8). Transferir pocos datos puede hacer imposible el realizar la tarea. Por ejemplo, si el módulo que hace la llamada no transfiere el número de identificación del proveedor, ¿cómo podría saber el módulo subordinado cuál es el registro que debe localizar?

También evite los diseños que crean datos flotantes. Esto ocurre cuando un módulo produce datos que no son necesarios para el módulo que hace la llamada sino para otro módulo del sistema. Los detalles se transfieren por todo el sistema (de ahí el término "flotante"), hasta que finalmente llegan a la función que los necesita. El rediseñar el sistema para establecer un acoplamiento holgado, así como la creación de módulos con más cohesión, evitarán esta dificultad.

Cohesión

El uso del enfoque descendente para planear el software de un sistema no garantiza que se evitarán los errores o que el sistema será fácil de mantener. En los sistemas modularizados adecuadamente y cohesivos, el contenido del módulo está diseñado para que lleve a cabo una

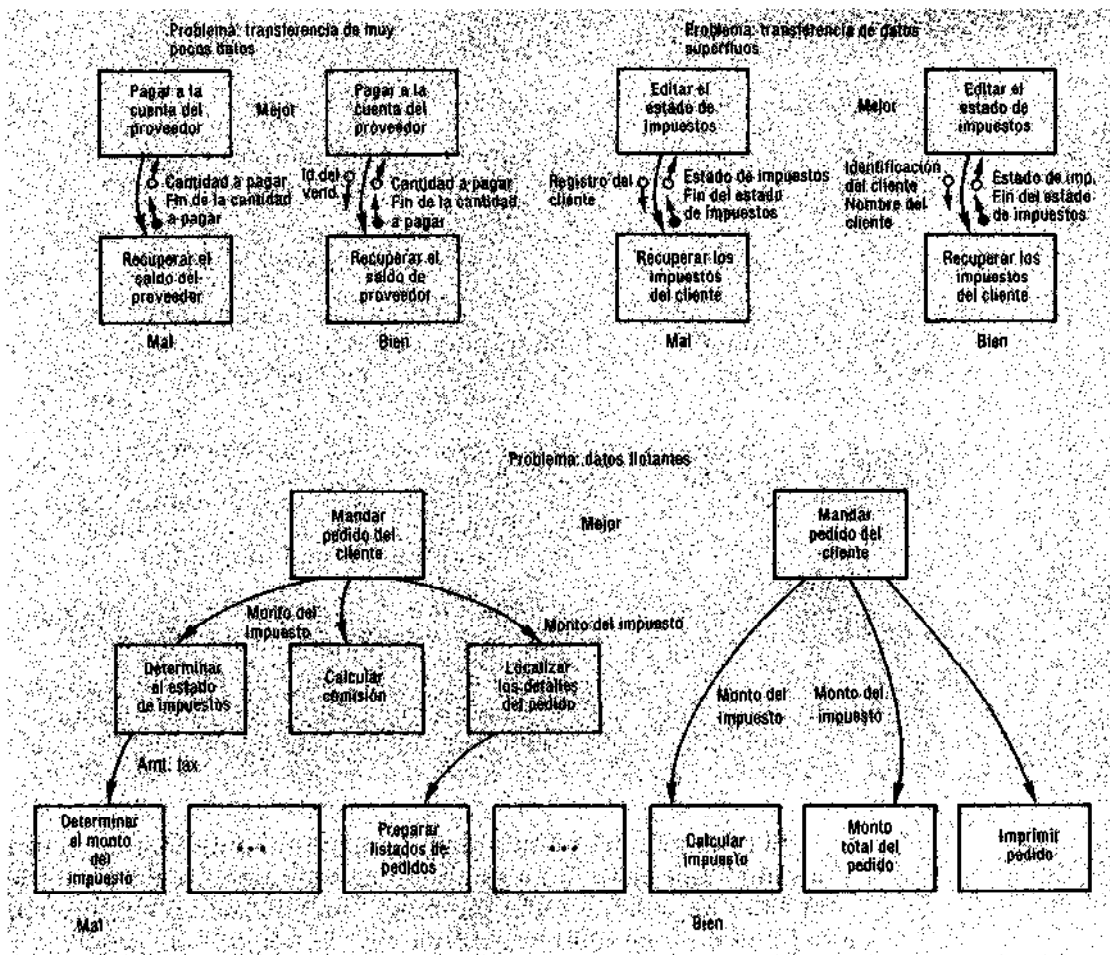


FIGURA 14.8
Ejemplos de diseño

función específica y para que sea más fácilmente entendible que los sistemas diseñados por otros métodos. Hay cuatro tipos generales de contenidos de módulos (Tabla 14.4); éstos se presentan enseguida en orden del menos al más deseable (Stevens, Myers y Constantine, 1974).

El tipo menos recomendable de agrupación del contenido de un módulo está formado por pasos que no llevan a cabo una función completa o que lógicamente no van juntos. Este caso extremo puede ocurrir si los programadores trabajan de acuerdo a reglas estrictas y dividen los módulos en secciones de 50 enunciados cada uno (o algún otro límite impuesto). Las actividades de entrada, salida, cálculo y manejo de archivos se llevan a cabo en un único módulo. Si esto ocurre, por lo general es el resultado del trabajo del programador sin especificaciones explícitas de diseño, o sin un claro entendimiento de cómo manejar una tarea.

	CONTENIDO DE UN MÓDULO	EXPLICACIÓN
MEJOR	Contenido de un módulo determinado por la función desarrollada	Todas las actividades en un módulo tienen el mismo propósito, es decir, llevar a cabo una función específica. <i>Ejemplos:</i> Calcular el cargo financiero en un estado de cuenta; editar una transacción.
	Contenido de un módulo determinado por los datos usados	Todos los elementos en un módulo se refieren a los mismos datos o archivos. <i>Ejemplos:</i> Imprimir, desplegar, y copiar datos de un archivo común.
	Contenido de un módulo determinado por la lógica del procesamiento	Todos los pasos se realizan juntos o manejan las mismas funciones. <i>Ejemplos:</i> Todas las operaciones de entrada, todas las operaciones de salida y todas las actividades de inicialización.
PEOR	Contenido no relacionado de un módulo	Los módulos se desarrollan por tamaño o número de instrucciones (es resultado de la programación con reglas escritas). <i>Ejemplos:</i> Todos los módulos no deben tener más de 50 enunciados de longitud; todos los módulos deben ajustarse a una sola página.

El contenido del módulo también se puede agrupar debido a que va junto lógicamente. Un módulo que maneja todas las operaciones de entrada o las de salida o uno que maneja actividades de procesamiento de pedidos, independientemente del tipo de cliente o las necesidades de manejo de datos por cada cliente, usa el agrupamiento lógico.

Los elementos del módulo también se pueden relacionar por el tiempo en el que ejecutan; es decir, lógicamente parecen ir juntos y se llevan a cabo al mismo tiempo. Un módulo que inicializa todas las variables y abre archivos está lógicamente integrado. Este nivel de modularidad es mejor que el primer tipo, ya que todos los elementos son ejecutables en un mismo marco de tiempo.

Los módulos que están lógicamente integrados son difíciles de modificar puesto que el costo será compartido por cada tipo de actividad. Hasta el más sencillo de los cambios puede afectar todos los tipos de transacción. Una mejor solución consiste en separar cada tipo de transacción en su propio módulo.

El contenido del módulo también se puede determinar por los datos que usen. Un módulo en el que el contenido se refiera a los mismos datos es preferible a uno que sea desarrollado sólo bajo la base de la lógica del procedimiento. Por ejemplo, se puede diseñar un módulo para que todas las operaciones sobre un conjunto de datos se

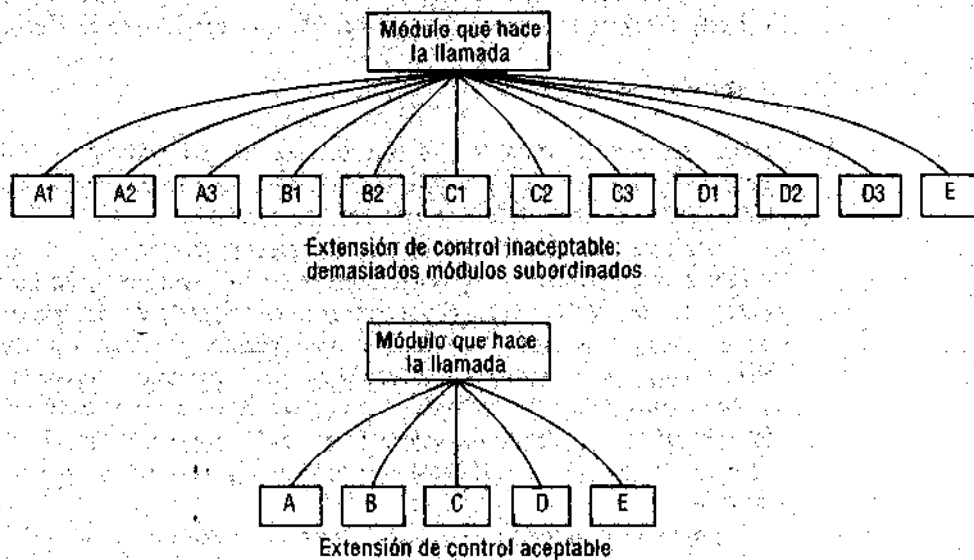


FIGURA 14.9
Extensión de control
en la jerarquía de un
módulo

Lleven a cabo al mismo tiempo: se prepara un archivo para imprimirse en papel, llevado a disco y también duplicado para fines de respaldo. Un módulo que lee la siguiente transacción y actualiza el archivo maestro, añadiendo, eliminando o cambiando registros, incluyendo la verificación de error necesaria para cada tipo de función, comparte un conjunto común de datos. Este tipo de cohesión es mejor que los otros tipos presentados, pero no tan bueno como el agrupamiento funcional.

El agrupamiento o cohesión funcional permite una prueba más global del módulo. Si se necesitan cambios posteriores, los analistas y programadores pueden determinar rápidamente cómo se estructura el módulo y cómo procesa los datos e interactúa con los demás módulos en el sistema.

Se enfatiza constantemente en la confiabilidad y facilidad de mantenimiento en todo el desarrollo del sistema.

Extensión de control

La *extensión de control* se refiere al número de módulos subordinados controlados por el módulo que hace la llamada. En general, hay que tratar de no tener más de cinco a siete módulos subordinados (Fig. 14.9).

Por otro lado, la excesiva extensión de control, lo que significa un alto número de módulos subordinados, crea un peso considerable cuando se desea determinar cuál módulo llamar bajo ciertas condiciones y al establecer secuencias de llamadas para transferir datos y

recibir resultados. Por otra parte, esto resulta común al no sujetarse a los objetivos de acoplamiento y cohesión presentados anteriormente.

Tamaño del módulo

¿Qué tan grande debe ser el módulo de un programa? Aunque es imposible fijar un número específico de instrucciones, existen algunos criterios útiles para definir el tamaño del módulo.

Algunas organizaciones han establecido reglas para el manejo del tamaño del módulo. Una regla común es que ningún módulo debe tener más de 50 instrucciones. Otra es que el listado del código fuente para un módulo debe limitarse a una página impresa. En algunas situaciones, estas reglas son adecuadas, pero en otras producen decisiones arbitrarias (por ejemplo, “si el módulo no puede codificarse en 50 instrucciones, crear un segundo módulo que sea llamado por el primero”) que restan coherencia a la decisión relativa al tamaño del módulo.

En general, debemos procurar diseños en donde los módulos tengan una función específica, sean altamente cohesivos y estén acoplados holgadamente. Sin embargo, los módulos no deben ser demasiado pequeños (cuando esto ocurre, deben conjuntarse los módulos). El tamaño del módulo depende también del lenguaje que se use. Más o menos, 50 enunciados en COBOL pueden ser un límite superior aceptable (los programadores no tienen que rediseñar si, digamos, se necesitan 51 o 55 enunciados, siempre que se cumplan los demás objetivos del diseño). Por otra parte, 50 instrucciones en un lenguaje poderoso de cuarta generación (donde una instrucción reemplaza de 10 a 50 enunciados equivalentes en COBOL) serían probablemente inaceptables.

Módulos compartidos

El uso compartido surge del deseo de tener una cierta función, cálculo o tipo de proceso llevado a cabo en un lugar del sistema. Se desea diseñar el sistema de forma que un cierto cálculo (como determinar los impuestos por ventas en un sistema de procesamiento de pedidos) se lleve a cabo una sola vez. Así, el módulo puede compartirse por todo el sistema llamándolo desde varios módulos.

¿Por qué compartir módulos? Hay varias razones. Primera, el compartir módulos minimiza la cantidad de software que debe diseñarse y escribirse. Segunda, minimiza el número de cambios que hay que hacer durante el mantenimiento del sistema. Por ejemplo, si cambian los procedimientos de cálculo de impuestos, solamente hay que modificar un módulo si se utiliza el principio de módulos compartidos. Y tercera, al tener un único módulo compartido se reduce la probabilidad de error: hay una probabilidad muy alta de que los módulos redundantes sigan distintos procedimientos de cálculo, si

bien no inicialmente, sino después de la introducción de cambios en el mantenimiento (un módulo puede no ser cambiado porque se haya perdido de vista).

Muchos sistemas establecen los módulos de biblioteca —procedimientos predefinidos— que se incluyen en la biblioteca de programas del sistema. La rutina se invoca rápidamente mediante un solo comando o llamada.

Los módulos de biblioteca compartidos son otra forma en la que los buenos diseñadores siguen los principios introducidos al principio de esta sección.

DISEÑO DE SOFTWARE Y HERRAMIENTAS DE DOCUMENTACIÓN

El software modular y bien diseñado tiene mayor probabilidad de cumplir con los requerimientos de facilidad de mantenimiento, confiabilidad y prueba delineados en el presente capítulo. En esta sección se estudian tres herramientas específicas: los diagramas de flujo estructurado, diagramas de HIPO y diagramas de Warnier/Orr.

Diagramas de flujo estructurado

Los *diagramas de flujo estructurado*, también llamados de *Nassi-Schneiderman*, son herramientas gráficas que fuerzan al diseñador a estructurar software que sea modular y descendente. Proporcionan una estructura a la que se pueden ajustar los que desarrollan el software de aplicación. Las responsabilidades en una organización varían. En algunas compañías, los analistas son responsables de desarrollar la lógica del módulo, mientras que en otras esa responsabilidad se delega en el programador. En cualquier caso, el programador debe estar preparado en el uso de los diagramas de flujo estructurado.

Elementos básicos

Existen tres elementos básicos usados en el desarrollo de los diagramas de flujo estructurado: proceso, decisión e iteración. (Hay muchas analogías entre estos elementos y los componentes que se usan en el español estructurado.)

Proceso Los procesos o pasos en un programa se representan mediante un rectángulo: el símbolo de proceso. Este símbolo representa la inicialización de variables, actividades de entrada y salida y las llamadas para ejecutar otros procedimientos.

Un nombre descriptivo breve, escrito dentro del rectángulo, establece el propósito del proceso. La serie de pasos se muestra usando varios rectángulos.

Decisión El símbolo de decisión representa condiciones alternativas que pueden ocurrir y que el programa debe poder manejar. Muestran el equivalente de las estructuras IF-THEN-ELSE, estudiadas con anterioridad, en el caso del español estructurado y que son comunes a varios lenguajes de programación. Como se verá en los ejemplos, el símbolo de decisión puede mostrar acciones para más de dos alternativas al mismo tiempo.

Iteración El símbolo de iteración representa los ciclos y repetición de operaciones *mientras* exista una condición dada o *hasta* que haya una condición. La forma del símbolo de iteración muestra claramente el alcance de la iteración, incluyendo todos los procesos y decisiones contenidos dentro del ciclo. La parte izquierda del símbolo representa la ruta de repetición a seguir hasta que las condiciones que controlan la iteración se satisfagan.

Uso de diagramas de flujo estructurado

Los diagramas de flujo estructurado no usan flechas o continuaciones en páginas separadas. Cada diagrama de flujo estructurado se muestra en una sola hoja de papel (o una sola pantalla, si se desarrolla en línea).

Al diseñar un diagrama de flujo estructurado, los analistas de sistemas especifican la lógica en una forma descendente. El primer elemento a considerar en un proceso o decisión es el elemento de arriba. El segundo en la serie es el siguiente que se muestre, y así sucesivamente. En forma análoga, existe una única salida del proceso.

El analista comienza con un proceso principal e introduce otros símbolos para subdividir el proceso. Cada proceso recibe un nombre, pero, si el nombre no se subraya, es una referencia a otro diagrama o descripción. Esta sencilla convención hace posible conjuntar fácilmente distintos procedimientos que se llevan a cabo para terminar una actividad compleja.

La figura 14.10 muestra los pasos necesarios para producir las facturas de los clientes durante el procesamiento mensual. El diagrama de flujo estructurado se lee de arriba hacia abajo y de izquierda a derecha. Cada actividad se anida dentro de la iteración y procesos alternativos del cual es parte. Además, se muestra claramente cada condición.

Algunas partes de los procesos a menudo se describen en diagramas de menor nivel. Por ejemplo, las transacciones de compra, pago, débito y crédito se señalan en el módulo que se describe en la figura 14.10, pero los módulos que manejan el procesamiento de cada tipo de transacción se aplican aparte.

Un uso de los diagramas de flujo estructurado importante para el diseñador que debe verificar las especificaciones del sistema contra la lógica planeada del software, es el de identificar las condiciones y los procedimientos a seguir cuando existan dichas condiciones. El hecho

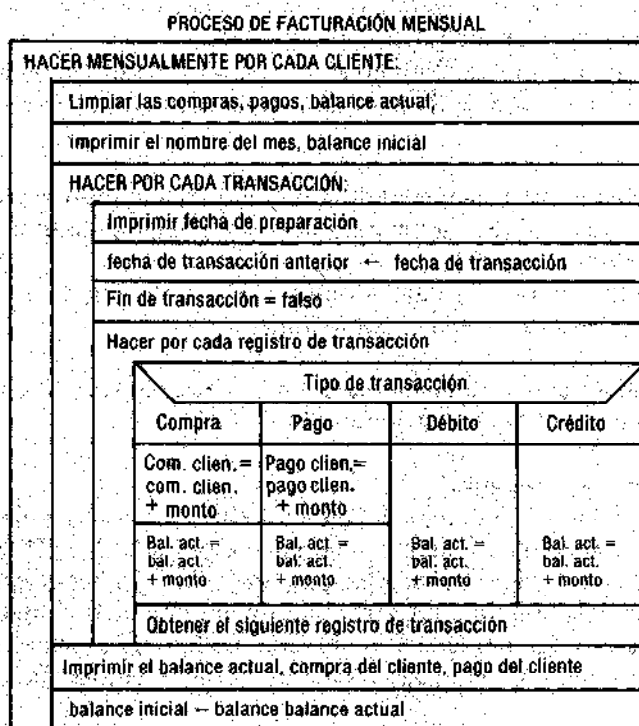


FIGURA 14.10
Diagrama de estructura para la facturación mensual

de que el diagrama de flujo estructurado sea fácil de leer permitirá al analista determinar si la transacción de ajuste de débito, por ejemplo, fue añadida por el programador o era parte de las especificaciones originales del sistema.

HIPO

HIPO es otro método de uso común para desarrollar software de sistemas. *HIPO* es una abreviatura del nombre en inglés de la *entrada-proceso-salida-jerarquica*, método que fue desarrollado por IBM para sus sistemas operativos grandes y complejos.

Propósito

La hipótesis en la que *HIPO* se basa es que es fácil perder la pista de la función deseada de un sistema o componente de un sistema grande. Esta es una razón por la que es difícil comparar los sistemas existentes contra sus especificaciones originales (y por lo tanto, porque pueden ocurrir fallas incluso en los sistemas técnicamente bien formulados). Desde el punto de vista del usuario, una sola función puede a menudo extenderse a varios módulos, por lo tanto, el interés del analista es entender, describir y documentar los módulos y su interacción de

forma que se obtenga el detalle suficiente, pero que no se pierda de vista el panorama general.

Los diagramas de HIPO son descripciones gráficas del sistema, en vez de prosa o narrativa. Ayudan al analista a responder tres preguntas guía:

1. *¿Qué hace el sistema o módulo?* (Se pregunta al diseñar el sistema.)
2. *¿Cómo lo hace?* (Se interroga al revisar el código para su prueba o mantenimiento.)
3. *¿Cuáles son las entradas y salidas?* (Se pregunta al revisar el código para su prueba o mantenimiento.)

Una descripción de HIPO para un sistema consta de una tabla visual de contenidos y los diagramas funcionales.

Tabla visual de contenidos

La *tabla visual de contenidos (VTOC)* muestra la relación entre cada uno de los documentos que conforma un paquete de HIPO. Está formada por un diagrama de jerarquía que identifica los módulos en un sistema mediante un número y en relación con los otros (Fig. 14.11) y da una descripción breve de cada módulo. Los números en la sección de contenido se relacionan con los de la sección de organización.

Los módulos aparecen en detalle creciente. Según la complejidad del sistema, son típicos de 3 a 5 niveles de módulos.

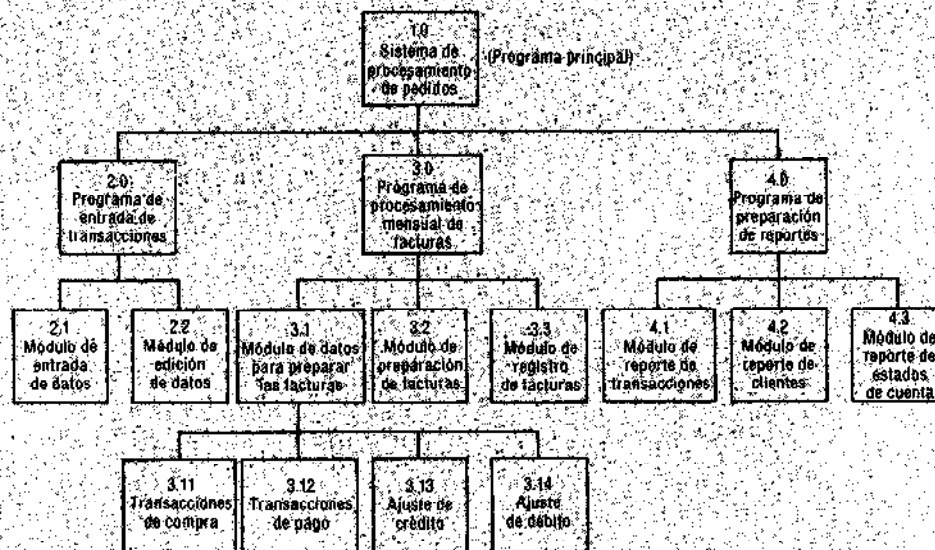
Diagramas funcionales

Existe un diagrama por cada caja en el VTOC. Cada diagrama muestra la entrada y salida (de derecha a izquierda o de arriba hacia abajo), los procesos principales, movimiento de datos y puntos de control. Los símbolos de los diagramas de flujo tradicionales representan los medios, tales como cinta magnética, disco magnético y salida impresa. Una flecha sólida muestra las rutas de control y una flecha hueca identifica el flujo de datos.

La ilustración de la figura 14.11 es un panorama del sistema, que representa la caja del nivel más alto en el VTOC. Los números de la parte inferior del diagrama indican donde se halla más detalle; es decir, se refieren a otros números de documento. Por ejemplo, el documento 3.11 explica el proceso de transacción de compra con mayor detalle.

Algunos diagramas funcionales contienen otros diagramas intermedios (Fig. 14.12). Pero también muestran datos externos, al igual que datos desarrollados internamente (como las tablas en el ejemplo de las facturas) y el paso en el procedimiento donde se usan los datos. Se le puede añadir una descripción de un diccionario de datos para explicar aún más los elementos de datos usados en un proceso.

Los diagramas de HIPO son efectivos para documentar un sis-



CONTENIDO

- | | |
|--|--|
| <p>1.0 SISTEMA DE PROCESAMIENTO DE PEDIDOS (PROGRAMA PRINCIPAL)
Controla todo el procesamiento. Llama programas para manejar la entrada de datos, el procesamiento mensual de los registros y la impresión de reportes.</p> <p>2.0 PROGRAMA DE ENTRADA DE TRANSACCIONES
Controla la entrada de todos los datos y la edición de los datos ya almacenados. Incluye las funciones de compra, pago y ajuste.</p> <p>2.1 MÓDULO DE ENTRADA DE DATOS
Lleva a cabo la entrada y validación de datos para comprar, pagos y ajustes de crédito o débito.</p> <p>2.2 MÓDULO DE EDICIÓN DE DATOS
Lleva a cabo la recuperación y edición de los datos de transacción previamente analizadas para comprar, pagos y ajustes de crédito o débito. Permite cambios a los datos o a la eliminación del registro de transacción.</p> <p>3.0 PROGRAMA DE PROCESAMIENTO MENSUAL DE FACTURAS
Controla todos los pasos del procesamiento de facturas usando los datos de transacciones introducidos durante el mes.</p> <p>3.1 MÓDULO DE DATOS PARA PREPARAR LAS FACTURAS
Lleva a cabo el procesamiento mensual de los datos de las transacciones para preparar la impresión de las facturas. Llama a módulos separados para manejar las transacciones de compra, pago y ajuste.</p> | <p>3.2 MÓDULO DE PREPARACIÓN DE FACTURAS
Imprime las facturas en formas ya impresas. Debe hacerse sólo hasta después de que los datos de la factura han sido preparados por el módulo 3.1.</p> <p>3.3 MÓDULO DE REGISTRO DE FACTURAS
Imprime una relación detallada de facturas numeradas en forma ascendente. Debe hacerse sólo hasta después de que los datos de la factura hayan sido preparados por el módulo 3.1.</p> <p>4.0 PROGRAMA DE PREPARACIÓN DE REPORTES
Controla toda la impresión de un reporte para los reportes de transacción, cliente y estado de cuenta.</p> <p>4.1 MÓDULO DE REPORTE DE TRANSACCIONES
Imprime un listado detallado de todas las transacciones introducidas durante el mes, ordenado según el número de transacción asignado.</p> <p>4.2 MÓDULO DE REPORTE DE CLIENTES
Imprime un listado de todos los clientes y la información demográfica en orden alfabético y de número de cuenta.</p> <p>4.3 MÓDULO DE REPORTE DE ESTADOS DE CUENTA
Imprime los listados de todas las cuentas con balance distinto de cero en orden descendente. Incluye la información cronológica.</p> |
|--|--|

FIGURA 14.11

Tabla visual de contenido HIPO.

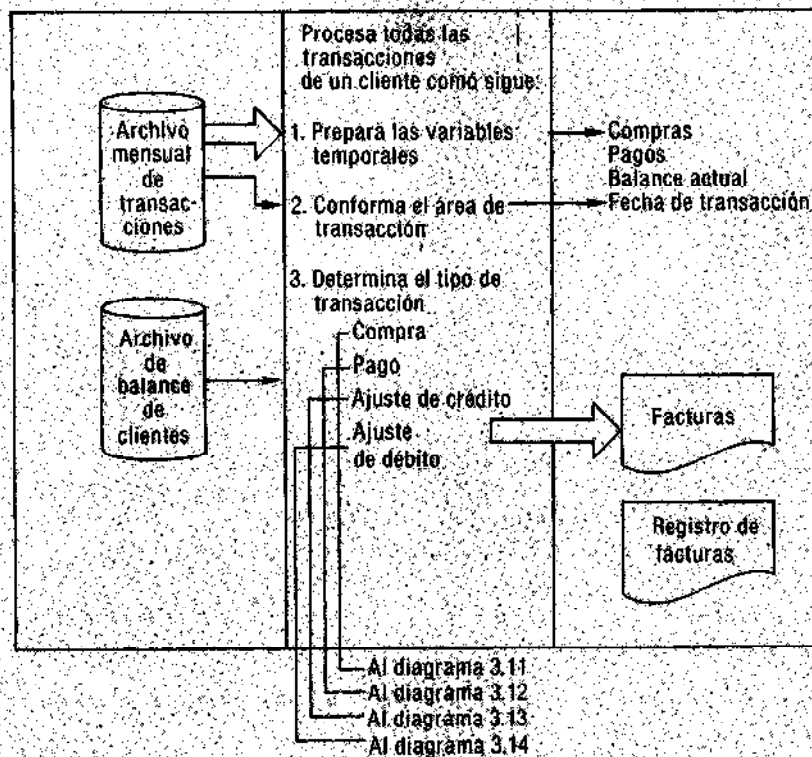


FIGURA 14.12
Diagrama funcional
de HIPO para el
procesamiento
mensual de facturas.

tema. También ayuda a los diseñadores y los fuerzan a pensar en cómo cumplir con las especificaciones y dónde hay que ligar las actividades y componentes. Sin embargo, se basan en un conjunto de símbolos especializados que requieren explicación, una preocupación adicional si se compara con la simplicidad de, por ejemplo, los diagramas de flujo de datos. Los diagramas de HIPO no son tan fáciles de usar para los fines de comunicación como mucha gente quisiera. Y, por supuesto, no garantizan sistemas libres de errores. Por lo tanto, su mayor fuerza es la documentación de un sistema.

Diagramas de Warnier/Orr

Los diagramas de Warnier/Orr (también conocidos como *construcción lógica de programas/construcción lógica de sistemas*) fueron desarrollados inicialmente en Francia por Jean-Dominique Warnier y en los Estados Unidos por Kenneth Orr. Este método ayuda al diseño de estructuras de programas identificando la salida y resultado del procedimiento, y entonces trabaja hacia atrás para determinar los pasos y combinaciones de entrada necesarios para producirlos. Los sencillos métodos gráficos usados en los diagramas de Warnier/Orr hacen evi-

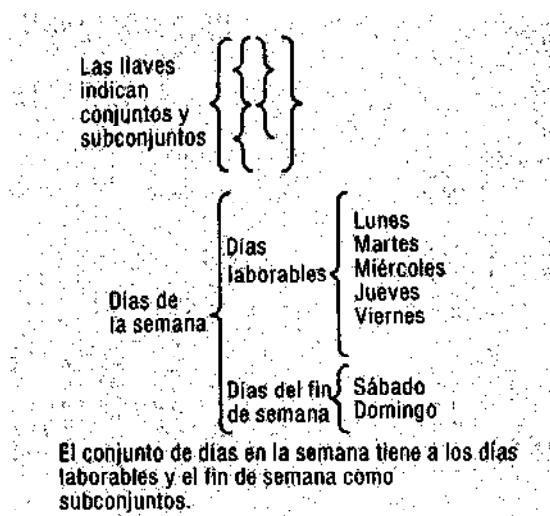


FIGURA 14.13
Simbología de conjuntos usada en los diagramas de Warnier/Orr.

dentes los niveles en un sistema y más claros los movimientos de los datos en dichos niveles.

Elementos básicos

Los diagramas de Warnier/Orr muestran los procesos y la secuencia en que se realizan. Cada proceso se define de una manera jerárquica; es decir, consta de conjuntos de subprocesos que lo definen. En cada nivel, el proceso se muestra en una llave que agrupa a sus componentes (Fig. 14.13). Puesto que un proceso puede tener muchos subprocesos distintos, un diagrama de Warnier/Orr usa un conjunto de llaves para mostrar cada nivel del sistema.

Los factores críticos en la definición y desarrollo del software son la iteración o repetición y la selección de alternativas. Los diagramas de Warnier/Orr muestran esto muy bien. Por ejemplo, al desarrollar un sistema de facturación, existen procesos que ocurren en cada transacción, cada día y cada mes. Como lo muestra la figura 14.14, la repetición de cada caso no sólo es fácil de identificar sino también se localiza rápidamente en una secuencia lógica.

En algunas situaciones, la única preocupación es si está presente cierta característica. Las alternativas son que esté o no esté. La simbología que se usa para indicar que una condición no existe es una línea sencilla sobre el nombre de la condición. Por ejemplo, en la figura 14.15 los cuatro tipos de transacción de compra, pago, ajuste del débito y ajuste del crédito son alternativas. El símbolo + representa alternativas.

Uso de diagramas de Warnier/Orr

La capacidad de mostrar la relación entre procesos y pasos de un proceso no es exclusiva de los diagramas de Warnier/Orr, así como

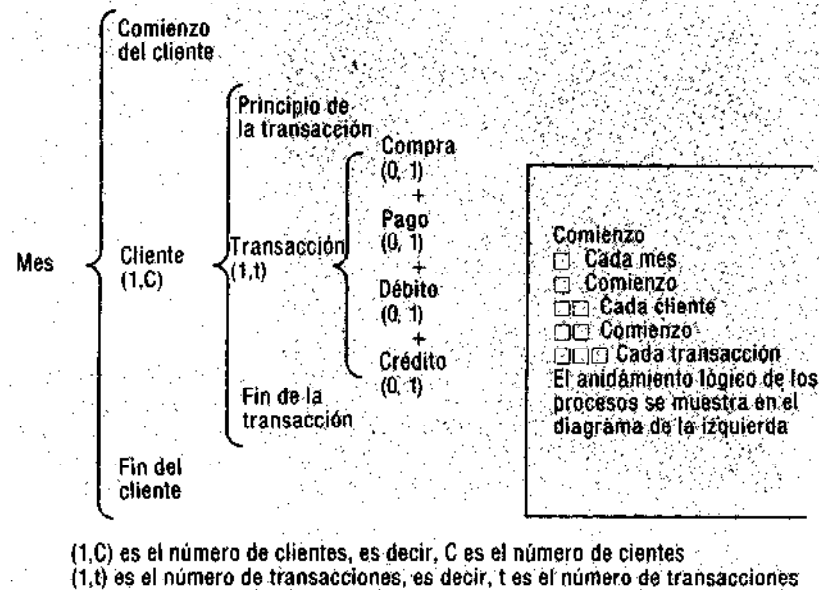


FIGURA 14.14
 Seguimientos en el
 procesamiento
 mensual de facturas

tampoco lo es el uso de la iteración, selección de alternativas o el tratamiento de casos individuales. Tanto los diagramas de flujo estructurado y los métodos del español estructurado logran esto también. Sin embargo, el enfoque que se usa para desarrollar las definiciones de un sistema por medio de diagramas de Warnier/Orr es distinto y se adapta bien a los que se usan en el diseño de sistemas lógicos.

Para desarrollar un diagrama de Warnier/Orr, el analista trabaja hacia atrás, empezando con la salida del sistema y usando un análisis orientado hacia la salida. En el papel el desarrollo se mueve de izquierda a derecha. En primer lugar, se definen la salida o resultados esperados del procedimiento. En el nivel siguiente, mostrado mediante la inclusión por medio de una llave, se definen los pasos necesarios para producir la salida. A su vez, cada paso se define un poco más. Las llaves adicionales agrupan los procesos requeridos para producir el resultado en el siguiente nivel.

Un diagrama completo de Warnier/Orr incluye las agrupaciones de procesos y los requerimientos de datos. Como lo indica la figura 14.15, los elementos de los datos se enlistan para cada proceso o componente del proceso. Estos elementos de datos son los necesarios para determinar cuál alternativa o caso se debe manejar por el sistema y llevar adelante el proceso. El analista tiene que determinar dónde se origina cada elemento dato, cómo se usa y cómo se combinan los elementos individuales. Al terminar la definición, se documenta una estructura de datos para cada proceso. Ésta, a su vez, es usada por los

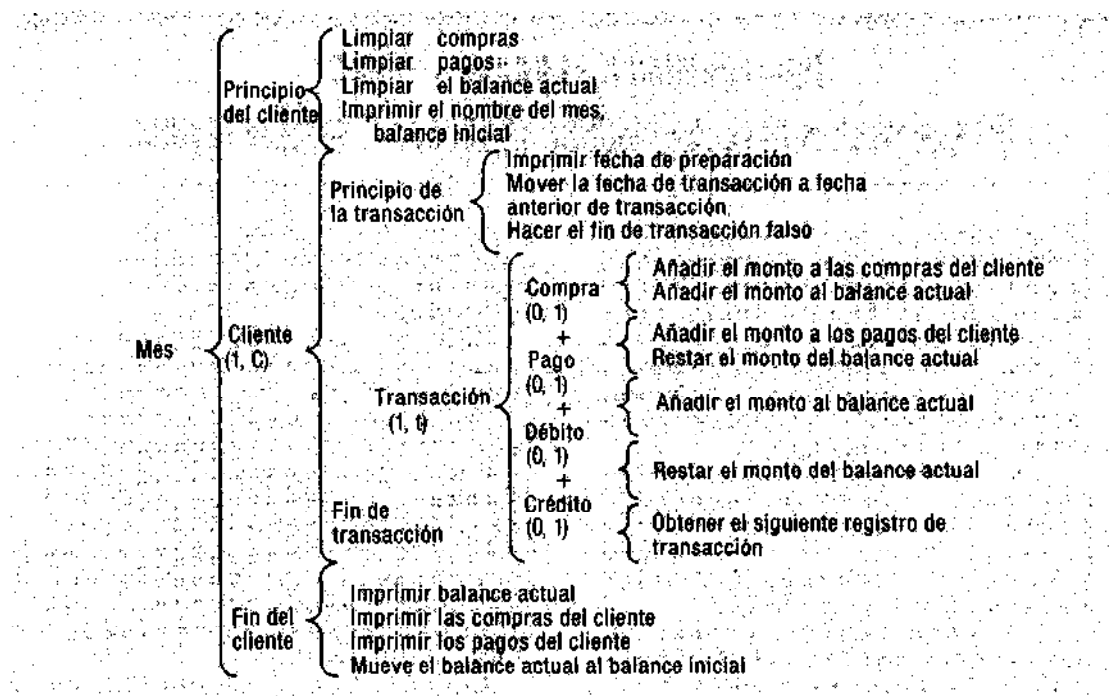


FIGURA 14.15

Datos que se usan en el procesamiento mensual de las facturas

programadores, los que trabajan a partir de los diagramas para codificar el software.

Los diagramas de Warnier/Orr ofrecen a los expertos en sistemas algunas ventajas distintivas. Son simples en apariencia y fáciles de entender. Aún así, son poderosas herramientas de diseño. Tienen la ventaja de mostrar agrupaciones de procesos y los datos que deben transferirse de nivel a nivel. Además, la secuencia del trabajo hacia atrás garantiza que el sistema estará orientado hacia el resultado. Este método también es un proceso natural. Con los diagramas de flujo estructurado, por ejemplo, a menudo es necesario determinar los pasos más internos antes de poder resolver lo relativo a las iteraciones y la modularidad.

Este método es útil para definir los datos y los procesos. Se puede usar para cada uno independientemente o se pueden combinar ambos en el mismo diagrama, como se hizo en el ejemplo anterior.

Una característica poderosa de los diagramas de Warnier/Orr es la forma en que se muestran los ciclos. Los pasos necesarios para moverse de un mes al otro en el procesamiento de facturas, por ejemplo, se señalan en la figura. Los pasos del procesamiento y los pasos de preparación de reportes se muestran con claridad, de tal forma que quien use el diagrama puede visualizar rápidamente cuándo se ejecutan los pasos y cuáles artículos están listos para terminar un ciclo

INGENIERÍA HACIA ATRÁS: REENCARNACIÓN DE UN SISTEMA

La mayoría de los aproximadamente 750 000 analistas de sistemas y programadores de los Estados Unidos trabajan en sistemas de información existentes. El mantenimiento de los sistemas de información es una enorme tarea en prácticamente cualquier organización. A nivel nacional, se invierten aproximadamente 55 billones de dólares al año para dar mantenimiento a los sistemas existentes.

Los analistas están prestando cada vez más atención al desarrollo de métodos, herramientas y técnicas para dar mantenimiento a los sistemas existentes. La ingeniería hacia atrás se está convirtiendo en un método valioso a considerar para llevar a cabo esto.

La ingeniería hacia atrás es el proceso de crear especificaciones, las cuales describan la acción de las aplicaciones existentes, en contraste con la ingeniería hacia adelante, en la que las especificaciones se crean de lo general a lo específico y que son seguidas por el desarrollo. La ingeniería hacia atrás también se puede llevar a cabo para rehacer la información que describe el propósito de los sistemas existentes.

La ingeniería hacia atrás es un concepto importante que se adecúa a la evolución de los sistemas de información. En la mayoría de las corporaciones, los sistemas no mueren; se les redirige, lo cual significa que los componentes se vuelven a desarrollar y usar para crear nuevas aplicaciones.

La documentación que describe en forma completa a los sistemas de información, así como los formatos de archivos y bases de datos, y el código fuente de los programas, es un cuerpo importante de detalles que explica cómo funciona un sistema, y en algunos casos, por qué se le ha diseñado de esa manera. Los comentarios en el código del programa y la

información que se incluye en reportes con cierto formato también proporcionan pistas importantes para hallar la lógica oculta en algunos programas.

Todo esto pudiera parecer misterioso, casi como introducirse por la puerta trasera de una casa solitaria. Y hay cierta verdad en eso. En muchas compañías, la persona o equipo que desarrolló el sistema puede haberse cambiado a otra empresa por lo que no está disponible para explicar por qué un sistema se desarrolló de cierta manera incluso cómo funciona. La ingeniería hacia atrás proporciona dicha información crítica para su uso en la modificación o rediseño del sistema o para encontrar formas para poder usar los componentes en otros sistemas.

No existen hasta el momento métodos estándares para llevar a cabo la ingeniería hacia atrás, aunque éstos llegarán algún día. Se espera que evolucionen las herramientas automatizadas para la ingeniería hacia atrás, permitiendo que los usuarios interactúen con un sistema para transferir el software en especificaciones. Es probable que un sistema de pregunta-respuesta permita consultas sobre, digamos, dónde se usa un cierto grupo de datos, dónde entra al sistema y en qué forma se genera. También es muy probable que ciertas herramientas produzcan bosquejos gráficos de la interrelación entre componentes, como un mapa de carreteras que une puntos clave.

La capacidad para ayudar a los analistas a desarrollar los sistemas con ingeniería hacia atrás podría convertirse en un importante elemento para mejorar la productividad del desarrollo de las aplicaciones nuevas y existentes.

mensual y prepararse para el siguiente. Este aspecto del procesamiento del sistema es crítico en un sistema de calidad.

MANEJO DEL PROCESO PARA GARANTIZAR LA CALIDAD

Las técnicas y criterios presentados en la sección anterior producirán, en caso de aplicarse adecuadamente, sistemas bien diseñados y con pocas fallas. Sin embargo, aun cuando las técnicas se sigan cuidadosamente, el analista no debe suponer que se ha cumplido con los estándares de calidad necesarios, punto que se enfatizó en el ejemplo del principio del capítulo.

El aseguramiento de la calidad es la revisión de los productos y documentación relacionada con el software para verificar su cobertura, corrección, confiabilidad y facilidad de mantenimiento. Y, por supuesto, incluye la garantía de que un sistema cumple las especificaciones y los requerimientos para su uso y desempeño deseados.

Niveles de seguridad

Los analistas usan cuatro niveles de aseguramiento de la calidad: prueba, verificación, validación y certificación.

Prueba

La prueba del sistema es un proceso caro pero crítico que puede llevarse hasta 50% del presupuesto para el desarrollo del programa. El punto de vista común respecto a las pruebas compartido por los usuarios, es que se lleva a cabo para demostrar que no hay errores en un programa. Sin embargo, como se indicó anteriormente, esto es prácticamente imposible, puesto que los analistas no pueden demostrar que el software está limpio de errores.

Por lo tanto, el enfoque más útil y práctico es en el entendimiento de que la prueba es el proceso de ejecutar un programa con la intención explícita de hallar errores, es decir, hacer que el programa falle. El examinador, que puede ser un analista, programador, o especialista entrenado en la prueba de software, está tratando realmente de hacer que el programa falle. Así, una prueba exitosa es aquella que encuentra un error.

Los analistas saben que un programa de prueba efectivo no garantiza la confiabilidad del sistema. La confiabilidad es asunto del diseño, como se estudió en este capítulo. Por lo tanto, la confiabilidad debe diseñarse en el sistema. Los analistas no pueden hacer una prueba de ella. Posteriormente, en esta misma sección, se discutirán estrategias específicas de prueba.



Comentario al margen

Estimación de la confiabilidad de los sistemas: el cliente tiene el control

En las empresas, un dicho común entre los coordinadores y los representantes de ventas es “el cliente siempre tiene la razón”. Últimamente, la frase, aunque no el significado, ha cambiado: “el cliente tiene el control”, lo cual quiere decir que el cliente determina lo que es correcto y aceptable.

Esto es similar en los sistemas de información. Los usuarios son los clientes y los usuarios tienen el control. Determinan si un sistema es bueno o no. Cualquier intento de un analista por convencer a los usuarios de lo sabio o correcto de un cierto enfoque no sirve de nada si los usuarios no piensan que el sistema funciona de manera razonable.

Cuando hay que decidir si un sistema es confiable, la decisión es muy simple. Si los usuarios —aquellos que usarán el sistema semana tras semana— no están convencidos que hace lo correcto (y en la forma correcta, sin sorpresas), no es un sistema confiable. Como dicen, ¡el cliente tiene el control!

Verificación y validación

Como la prueba, la *verificación* tiene la intención de hallar errores. Se lleva a cabo ejecutando un programa en un ambiente simulado. La *validación* se refiere al proceso del uso del software en un ambiente no simulado para hallar sus errores.

Cuando los sistemas comerciales se desarrollan con la intención explícita de distribuirlos a través de terceros para su venta, o comercializarlos por medio de oficinas de la propia compañía, primero deben pasar por la verificación, a veces llamada *prueba alfa*. La retroalimentación de la fase de validación generalmente produce cambios en el software para resolver los errores y fallas que se descubren. Se elige un conjunto de instalaciones usuarias que ponen a trabajar el sistema en un ambiente real. Estas instalaciones de *prueba beta* usan el sistema en las actividades cotidianas; procesan transacciones en directo y producen salidas normales del sistema. El sistema está a prueba en toda la extensión de la palabra, excepto que los usuarios están advertidos de que están usando un sistema que puede fallar. Sin embargo, las transacciones que se están procesando y las personas que usan el sistema son reales.

Es posible continuar la validación durante varios meses. En el curso de la validación del sistema, puede ocurrir una falla y el software será modificado. El uso continuo posiblemente produzca fallas adicionales y la necesidad de más cambios.

COMPANÍA DE AUDITORES CERTIFICADOS

123 Wall Street
New York, New York 10015

CERTIFICACIÓN DE SOFTWARE

Hemos revisado el paquete de servicio médico de la Medical Office Associates Company y elaborado un informe que describe nuestra revisión. Las observaciones principales de nuestro informe son que, en nuestra opinión, el paquete tiene la documentación adecuada tanto en el nivel del sistema como en el de la aplicación, lo cual permite a sus usuarios potenciales que entiendan el paquete. Proporciona controles internos de procesamiento, informes y archivos de datos que son adecuados para permitir el desarrollo de auditorías de acuerdo con los principios y estándares generalmente aceptados en los Estados Unidos.

El sistema procesa la información de contabilidad utilizando procedimientos que permiten a los usuarios producir informes que se ajustan a los principios de contabilidad que generalmente son aceptados en los Estados Unidos.

Estos reportes y sus anexos son para el paquete de servicio médico proporcionado por Medical Office Associates Company.

FIGURA 14.16
Informe de
certificación de
software

Certificación

La *certificación* del software es una garantía de lo correcto de un programa, su importancia va en aumento para las aplicaciones de sistemas de información. Existe una creciente dependencia de la compra o renta de software comercial en vez del desarrollo "en casa". Sin embargo, antes que los analistas deseen aprobar la adquisición de un paquete, a menudo requieren de la certificación del software por parte del fabricante o de un tercero sin prejuicios.

Por ejemplo, algunas empresas importantes de contabilidad están involucradas en la certificación de paquetes de software, para garantizar que realmente hace lo que el vendedor afirma que realiza y de una manera apropiada. Para certificar de esta forma el software, la empresa asigna a un equipo de especialistas que cuidadosamente examinan la documentación del sistema para determinar lo que afirma el vendedor que el sistema hace y cómo lo lleva a cabo. Entonces ellos prueban el software contra estas afirmaciones. Si no se encuentran serias discrepancias o fallas, certificarán que el software hace lo que la documentación afirma (Fig. 14.16). Sin embargo, no certifican que el software sea el paquete correcto para una cierta organización. Esta responsabilidad sigue siendo de la organización y su equipo de analistas.

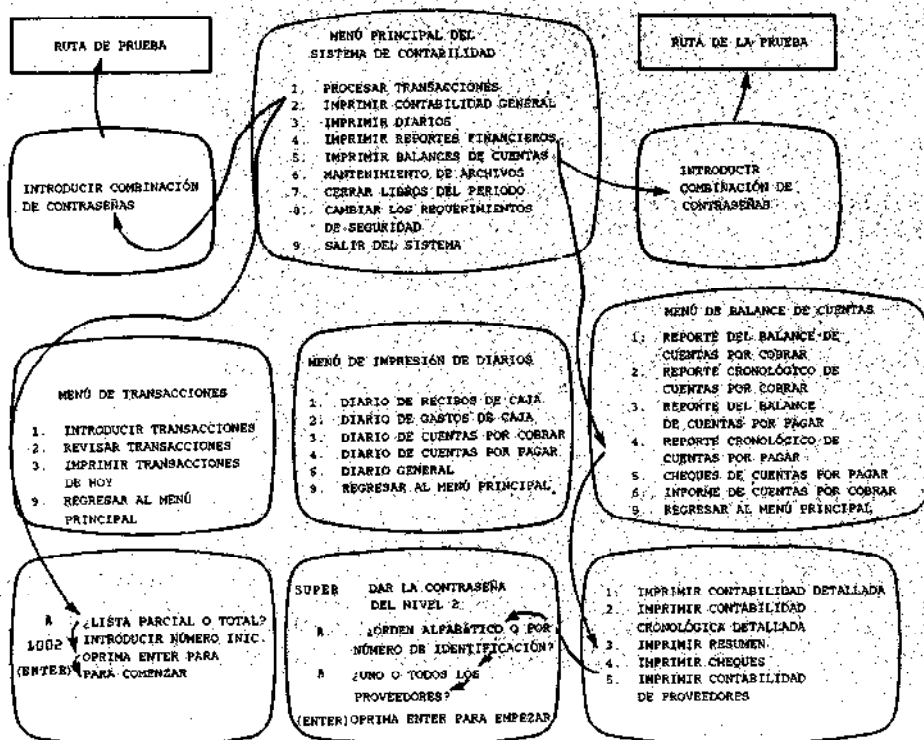


FIGURA 14.17

Datos para probar múltiples rutas en un sistema de contabilidad.

Estrategias de prueba

Ya se ha indicado que la filosofía detrás de la prueba es la de encontrar errores. Los casos de prueba se diseñan con este propósito en mente. Un *caso de prueba* es un conjunto de datos que el sistema procesará como entrada normal. Sin embargo, los datos se crean con la intención expresa de determinar si el sistema los procesará correctamente. Por ejemplo, los casos de prueba para el manejo de inventarios deben incluir situaciones en las que las cantidades que se retiran del inventario excedan, igualen y sean menores que las existencias. Cada caso de prueba se diseña con la intención de encontrar errores en la forma en que el sistema los procesará.

Hay dos estrategias generales para la prueba del software. Esta sección examina ambas, las estrategias de prueba de código y prueba de especificación.

Prueba de código

La estrategia de *prueba de código* examina la lógica del programa. Para seguir este método de prueba, el analista desarrolla casos de prueba que produzcan la ejecución de cada instrucción en el programa

o módulo; es decir, se prueba cada ruta del programa. Una ruta es una combinación específica de condiciones manejadas por el programa. Por ejemplo, en el caso del sistema de contabilidad, una ruta por el sistema es la de cambiar los balances de las cuentas. Se introduce la solicitud correcta, en seguida las contraseñas, datos y comandos adecuados (Fig. 14.17).

A primera vista, la prueba de código parece ser un método ideal para probar el software. Sin embargo, como vimos en el ejemplo del principio de este capítulo, es incorrecto el razonamiento de que todos los errores de software se pueden descubrir verificando toda ruta en un programa. En primer lugar, incluso en los programas moderadamente grandes, del tamaño usado en las situaciones típicas de un negocio, es casi imposible hacer una prueba exhaustiva de esta naturaleza. Las consideraciones financieras y las limitaciones de tiempo por lo general impedirán la ejecución de cada ruta dentro de un programa, ya que puede haber varios miles de ellas.

Sin embargo, aunque la prueba de código se pueda llevar a cabo en su totalidad, no es una garantía en contra de las fallas del software. Esta estrategia de prueba no indica si el código cumple sus especificaciones ni tampoco determina si todos los aspectos han sido implantados. La prueba de código tampoco verifica el rango de los datos que aceptará el programa, aun cuando, al ocurrir fallas del software en su uso real, sea frecuente que los usuarios introduzcan datos fuera de los rangos esperados (por ejemplo, una orden de ventas por un millón de dólares —la más grande en la historia de la organización—).

Prueba de especificación

Para llevar a cabo la *prueba de especificación* el analista examina las especificaciones que señalan lo que el programa debe hacer y cómo lo debe llevar a cabo bajo diferentes condiciones. Después se desarrollan casos de prueba para cada condición o combinación de condiciones y se mandan para su procesamiento. Por medio del estudio de los resultados, el analista puede determinar si el programa funciona de acuerdo con los requerimientos especificados.

Esta estrategia trata al programa como si fuera una caja negra: el analista no mira dentro del programa para estudiar el código y no le interesa si se prueba cada instrucción o ruta dentro del programa. En ese sentido, la prueba de especificación no es una prueba completa. Sin embargo, la hipótesis es que si el programa cumple las especificaciones, no fallará.

Ninguna de las estrategias de prueba de código o especificación es ideal. Sin embargo, la prueba de especificación es la estrategia más eficiente, ya que se centra en la forma que se espera se use el software. También muestra de nuevo qué tan importantes son las especificaciones desarrolladas por los analistas durante todo el proceso de desarrollo del sistema.

MANEJO DE LAS PRÁCTICAS DE PRUEBAS

Independientemente de cual estrategia siga el analista, hay ciertas prácticas preferidas para garantizar que la prueba sea útil. Los niveles de prueba y los tipos de datos de prueba, junto con las bibliotecas de prueba, son aspectos importantes del proceso real de prueba.

Niveles de prueba

Los sistemas no se diseñan como sistemas completos ni tampoco se prueban como sistemas únicos. El analista debe llevar a cabo tanto pruebas parciales como las del sistema.

Pruebas parciales

En las pruebas parciales, el analista prueba los programas que conforman a un sistema. (Por esta razón, a veces se llama a las pruebas parciales *prueba de programas*, en contraste con la prueba de sistemas, que se estudia en la siguiente sección.) Las unidades de software en un sistema son los módulos y rutinas que se ensamblan e integran para llevar a cabo una función específica. En un sistema grande, se necesitan muchos módulos en varios niveles.

Las pruebas parciales se centran primero en los módulos, independientes entre sí, para localizar los errores. Esto permite al que realice la prueba detectar errores en el código y lógica contenidos dentro de ese único módulo. Aquellos errores que resultan de la interacción entre los módulos se evitan inicialmente.

Por ejemplo, un sistema de información de un hotel está formado por módulos para manejar las reservaciones; entrada y salida de huéspedes, restaurant, servicio al cuarto y cargos varios, actividades de convención y elaboración de estados de cuenta. Para cada uno, se tiene la capacidad de introducir, cambiar o recuperar datos y responder a consultas o imprimir reportes.

Los casos de prueba necesarios para las pruebas parciales deben probar cada condición u opción. Por ejemplo, los casos de prueba se necesitan para determinar cómo maneja el sistema los intentos, para registrar a clientes que tengan o no reservación, lo mismo que aquellos casos en los que hay que cambiar el nombre en la reservación cuando llega una persona distinta a la esperada. También se necesitan casos de prueba para las situaciones en las que el cliente paga el monto exacto de la factura, sólo parte de ella o más del monto mostrado. Es más, debe incluirse en un caso de prueba la salida del cliente sin que realice pago alguno.

Si el módulo recibe una entrada o genera una salida, también se necesitan casos de prueba para examinar el rango de valores esperado, incluyendo los datos válidos e inválidos. ¿Qué ocurrirá en el ejemplo de la salida del cliente del hotel si un cliente desea hacer un pago de \$ 150 000 para una próxima convención? ¿Están diseñados los módu-

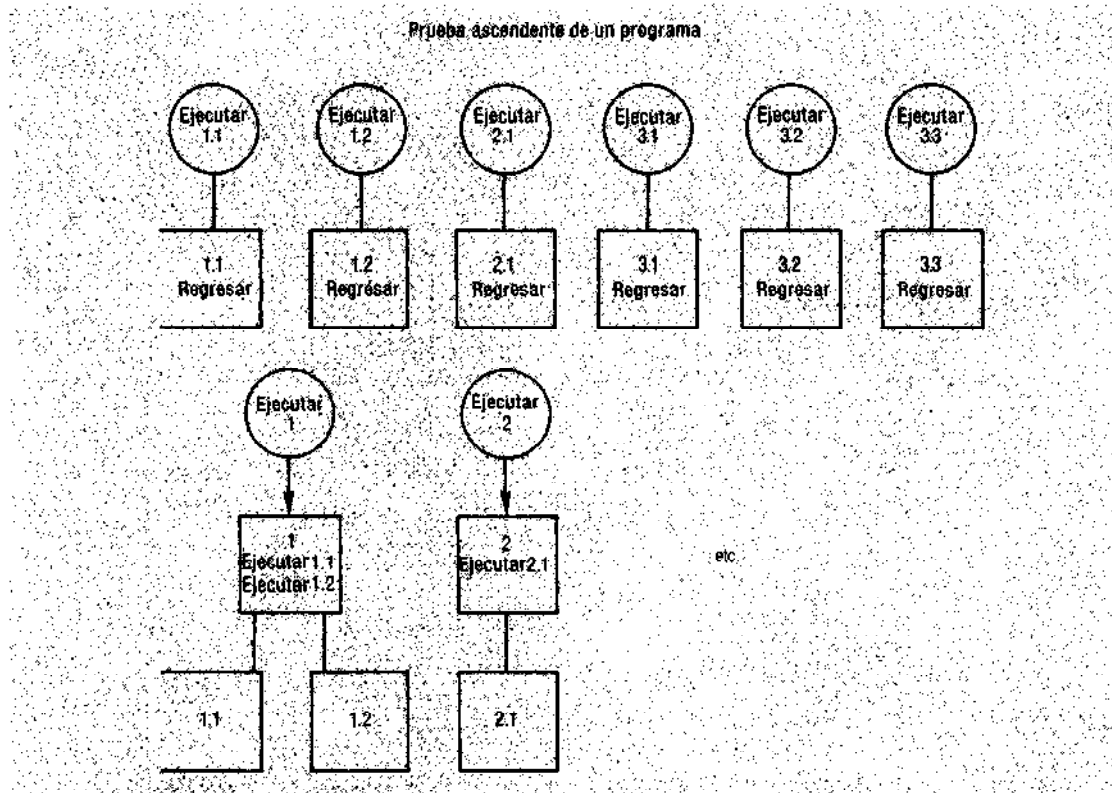


FIGURA 14.18
Método alternativo
para las pruebas
parciales usando un
enfoque ascendente

los de pago e impresión para manejar este monto? La prueba para esta pregunta detecta rápidamente los errores existentes.

Si el módulo se diseña para llevar a cabo iteraciones, con procesos específicos contenidos dentro de un ciclo, es recomendable ejecutar cada condición frontera: cero iteraciones, una iteración en el ciclo y el máximo número de iteraciones en el ciclo. Por supuesto, siempre es importante examinar los resultados de la prueba, pero hay que prestar especial atención a estas condiciones. Muy a menudo, los analistas cometen el error de suponer que el caso de cero iteraciones será manejado automáticamente en forma adecuada.

Las pruebas parciales se pueden llevar a cabo en forma ascendente, comenzando con los módulos más pequeños y de nivel inferior y continuando de uno en uno. Para cada módulo en la *prueba ascendente*, un programa corto (llamado *programa conductor* ya que maneja o corre el módulo) ejecuta el módulo y proporciona los datos necesarios, de esta forma se pide que el módulo se desempeñe en la forma en que lo haría al encajarse dentro del sistema. Después de probar los módulos de nivel inferior, la atención se centra en los del siguiente nivel (Fig. 14.18) que usan los de nivel inferior. Se prueban en forma

Prueba descendente de un programa.

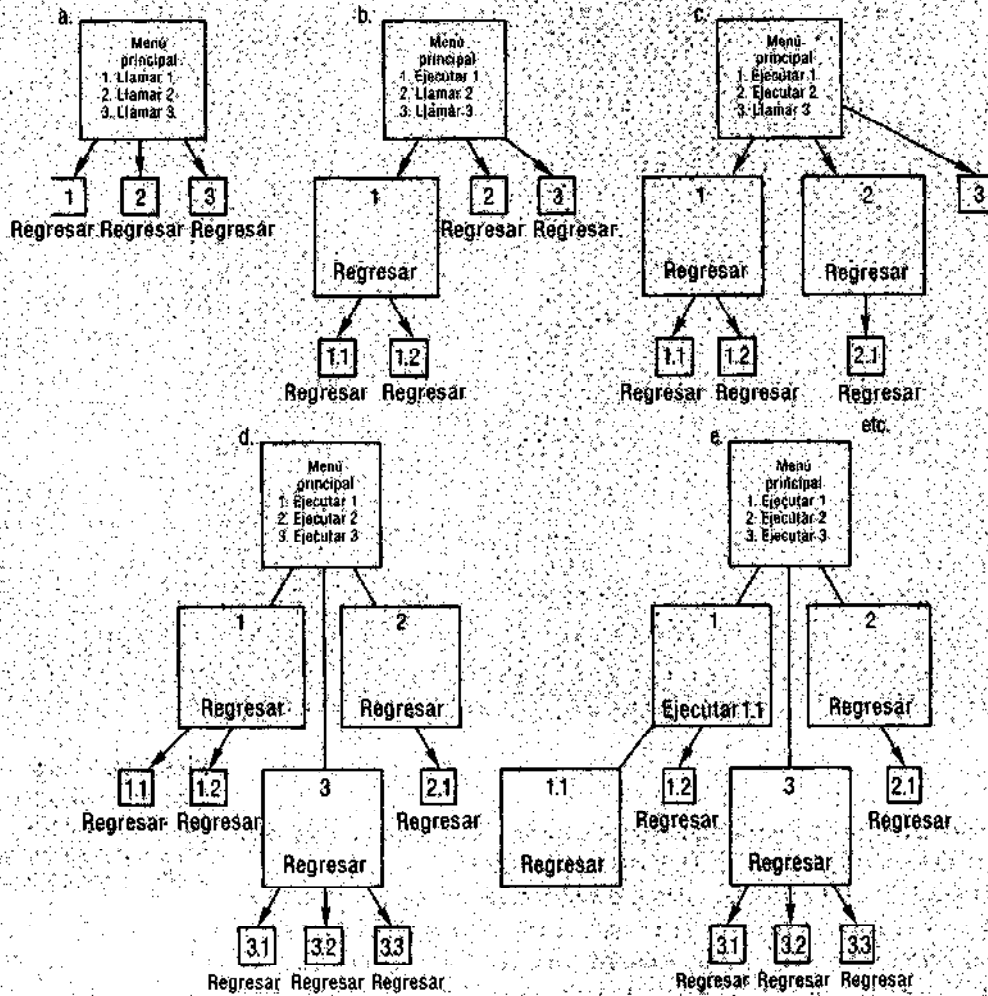


FIGURA 14.19

Método alternativo de prueba usando un enfoque descendente

individual y después conjuntamente con los módulos de nivel inferior que fueron examinados con anterioridad.

La *prueba descendente*, como su nombre lo indica, empieza con los módulos de nivel superior. Sin embargo, puesto que no se cuenta con las actividades de detalle, las que usualmente se llevan a cabo en las rutinas de nivel inferior (ya que esas rutinas no se están probando), se escriben fragmentos. Un fragmento es un módulo que puede ser llamado por el módulo de nivel superior y que, al ser alcanzado en forma apropiada, regresará un mensaje al módulo que hace la llamada, indicando que ocurrió la interacción apropiada (Fig. 14.19). No se hace el intento por verificar si el módulo del nivel inferior es correcto.

TABLA 14.3 Pruebas especiales de sistemas

TIPO DE PRUEBA	DESCRIPCIÓN
Prueba de carga máxima	Determinar si el sistema manejará el volumen de actividades que ocurran cuando el sistema esté en el punto más alto de su demanda de procesamiento. <i>Ejemplo:</i> Todas las terminales se activan al mismo tiempo.
Prueba de almacenamiento	Determinar la capacidad del sistema para almacenar datos de transacciones en un disco u otros archivos. <i>Ejemplo:</i> Verificar la documentación en el sentido de que el sistema almacenará 10 000 registros de 383 bytes de longitud en un único disco flexible.
Prueba de tiempo de ejecución	Determinar el tiempo de máquina que el sistema necesita para procesar los datos de una transacción. <i>Ejemplo:</i> Tiempo de respuesta a las consultas cuando el sistema está cargado en su totalidad con datos operativos.
Prueba de recuperación	Determinar la capacidad del usuario para recuperar los datos o restablecer el sistema después de una falla. <i>Ejemplo:</i> Cargar copias de respaldo de datos y reiniciar el procesamiento sin perder datos o integridad.
Prueba de procedimientos	Determinar la claridad de la documentación en los aspectos de operación y uso de un sistema, haciendo que los usuarios lleven a cabo exactamente lo que el manual pide. <i>Ejemplo:</i> Apagar el sistema al final de la semana o responder al indicador luminoso de la impresora que señala la falta de papel.
Prueba de factores humanos	Determinar como utilizarán los usuarios el sistema al procesar datos o preparar informes. <i>Ejemplo:</i> Reacciones del usuario cuando no hay una respuesta inmediata a una consulta.

A menudo, los planes de prueba descendente se conjuntan con las pruebas ascendentes; es decir, se prueban como unidad los módulos de nivel inferior y se integran a un programa de prueba descendente.

Prueba de sistemas

La prueba de sistemas no prueba el software en sí, sino la integración de cada módulo en el sistema. También busca las discrepancias entre el sistema y su objetivo original, especificaciones y documentación del sistema. La preocupación principal es la compatibilidad de los módulos individuales.

Los analistas tratan de hallar áreas en donde los módulos hayan sido diseñados con especificaciones distintas para la longitud y tipo de datos y los nombres de los elementos de los datos. Por ejemplo, un

módulo puede esperar que el tipo de dato para la identificación de un cliente sea un campo numérico, mientras que otros módulos esperan que sea de tipo carácter. Es posible que el sistema en sí no reporte esto como un error, pero la salida puede mostrar resultados inesperados. Si un registro creado y guardado en un módulo, usando la identificación como tipo numérico, se busca posteriormente esperando que sea de tipo carácter, el campo no será reconocido y aparecerá el mensaje EL REGISTRO PEDIDO NO SE ENCUENTRA.

La prueba de sistemas también debe verificar que los tamaños de los archivos son adecuados y que los índices se han construido en forma adecuada. Se deben probar a nivel sistema los procedimientos de ordenamiento y reindexación, que se supone están presentes en los módulos de nivel inferior, para ver que realmente existen y que logran los resultados que esperan los módulos.

Pruebas especiales de sistemas

Existen otras pruebas en una categoría especial ya que no se centran en el funcionamiento normal del sistema. Hay 6 pruebas básicas (Tabla 14.5).

Prueba de carga máxima Existen tiempos críticos en muchos sistemas, particularmente en los sistemas en línea. Por ejemplo, en un sistema bancario, los analistas quieren saber que ocurriría si todos los cajeros prenden sus terminales al mismo tiempo antes de empezar las actividades del día. ¿Los manejará el sistema uno a la vez sin incidentes?, ¿tratará de manejarlos todos a la vez y se confundirá tanto que se “cierre” y deba ser arrancado de nuevo?, ¿se perderán las direcciones de las terminales? La única forma segura para descubrirlo es probarlo. Las mismas situaciones pueden surgir cuando los cajeros apagan la terminal durante los descansos para comer y al final del día, por lo que la prueba se refiere a situaciones reales.

Prueba de almacenamiento Los analistas especifican una capacidad del sistema cuando se diseña y elabora. La capacidad se mide en términos del número de registros que un disco puede manejar o que un archivo puede contener. Estas capacidades están ligadas al espacio en disco y al tamaño de los índices, claves de registro y demás. Pero éstos también deben ser probados. Si la documentación de un sistema nuevo que va a correr en una microcomputadora afirma que un archivo en disco almacena más de 10 000 registros, cada uno con 393 bytes de longitud, la afirmación debe ser verificada antes de la implantación.

La prueba de almacenamiento requiere almacenar continuamente datos (ver las notas que siguen sobre los datos reales contra los sintéticos) hasta que se alcanza la capacidad. Al comparar las capacidades ofrecidas y reales se verificará, por un lado, la exactitud de la documentación y permitirá al mismo tiempo dar un juicio acerca de la

capacidad real. La mayoría de los sistemas no se prueban de esta forma. Los usuarios encuentran demasiado tarde que las afirmaciones hechas durante la instalación no son ciertas: no existe la capacidad suficiente de almacenamiento para las transacciones y los registros del archivo maestro.

Prueba del tiempo de ejecución Cuando los analistas están desarrollando un diseño, se preocupan más por los reportes, entradas, archivos y secuencias de proceso que por el tiempo de ejecución, aunque esto cambia con la experiencia. Durante las pruebas parciales y del sistema, se usan conjuntos relativamente pequeños de datos para hallar errores o provocar fallas. Por lo tanto, los usuarios con frecuencia saben qué tan rápido o lento es el sistema sólo hasta después de que ha sido instalado y cargado con los datos. Esto puede ocurrir demasiado tarde. (Rara vez, los sistemas son muy rápidos para los usuarios.)

La prueba del tiempo de ejecución se lleva a cabo antes de la implantación para determinar cuánto tiempo se lleva recibir una respuesta a una consulta, hacer una copia de respaldo de un archivo o mandar una transmisión y recibir una respuesta. También incluye corridas de prueba para conocer el tiempo necesario para indexar o reordenar grandes archivos del tamaño de los que tendrá el sistema durante una corrida típica, o bien preparar un reporte.

Un sistema que corre bien con sólo algunas transacciones de prueba puede ser inaceptablemente lento cuando esté cargado por completo. El momento para saber esto es antes de la implantación, cuando se pueden hacer con mayor facilidad los ajustes. Una vez que los archivos están cargados en su totalidad y el usuario confía en el sistema para sus actividades cotidianas, es difícil retirarlo y comenzar cambios a gran escala. El usuario necesita el sistema y el analista no quisiera arriesgar la pérdida de datos reales.

Prueba de recuperación Los analistas deben suponer siempre que el sistema fallará y que los datos se dañarán o perderán. Aunque se escriban planes y procedimientos para cubrir estas situaciones, también se deben probar. Mediante la creación de un evento de falla o pérdida de datos, en donde los usuarios se vean forzados a volver a cargar y recuperar a partir de una copia de respaldo, los analistas pueden determinar fácilmente si los procedimientos de recuperación son adecuados. Generalmente, aun los planes mejor diseñados se ajustan o aumentan después de esta prueba.

Prueba de procedimientos Los manuales de documentación y ejecución que dicen al usuario como llevar a cabo ciertas funciones se prueban fácilmente pidiendo al usuario que los siga en forma exacta por medio de una serie de eventos. Es sorprendente cómo pueden surgir preguntas ante el hecho de no incluir instrucciones sobre

cuándo oprimir la tecla ENTER, cuándo quitar los diskettes antes de apagar la máquina o qué hacer cuando se prende la luz que indica la falta de papel en la impresora.

Por supuesto, no hay sustituto para un conjunto de manuales de procedimiento bien diseñados. Los analistas se concentran en los detalles principales y críticos del diseño de un sistema y los incluyen en la documentación. También ponen atención en los pequeños detalles, tales como los mencionados anteriormente, al diseñar el sistema. Pero es común que las descripciones de los detalles no estén dentro de la documentación. Este tipo de prueba no sólo muestra dónde se necesitan, sino también en qué lugar están equivocados, es decir, dónde las acciones sugeridas en la documentación no son compatibles con las que realmente hay que llevar a cabo para hacer que el sistema funcione.



Comentario al margen

Documentación: para preservar la credibilidad

Lo peor que puede ocurrir cuando se implanta un nuevo sistema es que el software no funcione. Los profesionales de los sistemas de información se preocupan mucho por esta posibilidad, como se ha señalado en este capítulo.

Sin embargo, algo casi tan malo es que en la documentación los usuarios lean una cosa y que el sistema haga otra. En ese momento, tanto la aplicación como el analista pierden credibilidad. (Y una vez perdida la credibilidad es difícil de volverse a ganar.)

La documentación que va a dar a las manos del usuario final, ya sea en la forma de manuales, sistemas de ayuda en línea o diagramas sencillos, debe coincidir con la forma en que funcione la aplicación. Los analistas tienen que asegurarse de que ponen la atención suficiente para mantener este aspecto de credibilidad.

Prueba de los factores humanos ¿Qué hacen los usuarios si, después de mandar una transacción por medio de una terminal, la pantalla se pone en blanco mientras que los datos se procesan? Podrían no llevar a cabo las acciones que el analista desea o espera y en vez de ello responder de manera inusual: pueden oprimir la tecla de envío varias veces, apagar la terminal y volverla a prender, desconectar el teléfono y volverlo a conectar, volver a llamar al centro de cómputo o golpear la terminal. Obviamente, ellos harían cualquier cosa si el analista no les diera algún mensaje en la pantalla para indicar que su petición ha sido recibida, que está siendo procesada y que tardará un poco. De esto trata la prueba de factores humanos —hallar respuestas a preguntas sobre como reaccionará la gente ante el sistema en formas no previstas—. Como regla general, aunque parezcan extrañas las accio-

nes anteriores, la gente tiene la razón, llevan a cabo acciones que son normales bajo las circunstancias.

Es responsabilidad del analista el anticipar preguntas que surgirán en la mente de los usuarios cuando éstos interactúen con el sistema. Si una pantalla se pone en blanco durante el procesamiento de la transacción, el analista debe asegurarse de que aparezca un mensaje en el cual se informe al usuario que la computadora está trabajando. Incluso no es suficiente si el retraso es de más de un segundo o dos. Para el procesamiento que se tarde periodos largos, el analista debe hacer que la pantalla dé al usuario un mensaje en el que se dice aproximadamente *cuánto tiempo* se tardará y dando *una opción para cancelar la petición*. El usuario puede decidir correr ese trabajo de una hora en otro momento, cuando el sistema no esté tan ocupado.

Si el sistema va a hacer un proceso largo de ordenamiento, el analista debe mantener al usuario informado acerca de que proporción del ordenamiento ha terminado. Los usuarios aprecian los sistemas que muestran el número de registros ordenados o el porcentaje terminado.

También el analista debe asegurarse de *observar* cómo introducen los datos las personas. ¿Usan teclas diferentes de las anticipadas (tales como la parte superior de los números en el teclado de la máquina de escribir en vez del teclado numérico)? ¿Hay combinaciones difíciles de teclas que puedan provocar errores (por ejemplo, tener que mantener oprimida la tecla de mayúsculas con el dedo meñique mientras se presiona la tecla + con el índice)?

¿Cómo se sentirá el usuario de un sistema después de trabajar con él durante mucho tiempo? El resplandor de una pantalla o simplemente bastante detalle en la misma provoca irritación física y mental. Las ligeras modificaciones en el contenido de la pantalla o la distribución del equipo son preocupaciones importantes del factor humano que afectan de forma dramática al usuario y, por lo tanto, al sistema.

Estas preguntas simples de prueba son de considerable importancia y extremadamente útiles para hallar defectos que puedan provocar la falla del sistema. Algunos analistas hallarán estos defectos en la forma más difícil —por medio de malas experiencias—. No es fácil olvidar al sistema que fue dañado porque el usuario golpeó la terminal cuando introdujo los datos y fueron aceptados por el sistema sin mostrar una respuesta. Sin embargo, si sigue los criterios anteriores, el analista puede evitar dichas situaciones.

Diseño de datos de prueba

Hay dos fuentes muy diferentes de datos de prueba, reales y artificiales. Ambos tienen ventajas y desventajas para el que realiza la prueba.

Uso de datos de prueba reales

Los datos de prueba reales son aquellos que se extraen de los archivos

de la organización. Después de que un sistema está terminado parcialmente, es frecuente que los programadores o analistas pidan a los usuarios que introduzcan un conjunto de datos de sus actividades normales. Por ejemplo, en un sistema de contabilidad, le pueden pedir a alguien del personal de contabilidad que introduzca la relación de cuentas y los estados financieros, junto con transacciones que afecten a estas cuentas. El analista de sistemas usa estos datos para probar parcialmente el sistema. En otros casos, los programadores o analistas obtienen datos reales de los archivos y los introducen ellos mismos.

Es difícil obtener datos reales en cantidad suficiente para conducir una prueba extensa. Y, aunque los datos reales mostraran cómo funciona el sistema para los requerimientos típicos de procesamiento, suponiendo que los datos reales introducidos son en realidad típicos, generalmente dichos datos no probarán todas las combinaciones o formatos que puedan entrar al sistema. El sesgo hacia los valores típicos no proporciona entonces una verdadera prueba del sistema y de hecho ignora los casos más probables que causan la falla del mismo.

Uso de datos de prueba artificiales

Los datos de prueba artificiales se crean solamente con fines de prueba, ya que se pueden generar para probar todas las combinaciones de formatos y valores. En otras palabras, los datos artificiales, que se puedan preparar rápidamente mediante un programa de utilería para generación de datos en el departamento de sistemas de información, hacen posible la prueba de todas las rutas lógicas y de control en todo el programa.

Los programas de prueba más efectivos usan datos de prueba artificiales generados por personas distintas de las que escribieron los programas. Frecuentemente, un equipo independiente formula un plan de prueba, usando las especificaciones del sistema.

Bibliotecas de prueba

Para garantizar que todos los sistemas se prueban adecuadamente, muchas organizaciones establecen bibliotecas de prueba. Una *biblioteca de prueba* es un conjunto de datos desarrollados para probar en su totalidad un sistema. Se guarda en una forma fácil de leer por la máquina, usualmente en disco magnético, y se usa por todas las personas involucradas con un sistema particular.

Por ejemplo, un sistema grande de inventarios está formado por cientos de programas. Todos comparten datos y formatos de archivo comunes. Cada uno de ellos también procesará transacciones similares y a veces actualizará registros y en otras ocasiones recuperará datos para responder a consultas o prepara reportes y documentos. Puesto que estos programas son transacciones interdependientes y sus

procesos están relacionados, tiene sentido usar un conjunto común de datos para probar cada programa.

Las bibliotecas de prueba no solamente son para las pruebas iniciales. Al evolucionar el sistema y modificarse y dar mantenimiento a los programas, deben volverse a probar. La biblioteca de prueba tiene que conservarse durante toda la vida de un sistema de forma que, al hacer cada cambio, se disponga nuevamente de datos confiables para probar el sistema.

RESUMEN

La calidad de un sistema de información depende de su diseño, desarrollo, prueba e implantación. Un aspecto de la calidad del sistema es su *confiabilidad*. Un sistema es confiable si, al usarse de manera razonable, no produce fallas peligrosas o costosas. Esta definición distingue entre los *errores* del software, en los que el sistema no produce los resultados esperados, y las *fallas* que se presentan. Aunque es prácticamente imposible desarrollar software que se pueda demostrar que está libre de errores, los analistas hacen lo posible por prevenir la aparición de errores, usando métodos y técnicas que incluyen la detección de errores y su corrección y la tolerancia a errores. Ambas estrategias son útiles para mantener al sistema en operación y prevenir las fallas. A diferencia del hardware, en el que puede haber fallas de fabricación y del equipo, las fallas del software son el resultado de errores de diseño introducidos cuando se formularon las especificaciones y se escribió el software.

Un aspecto adicional del *aseguramiento de la calidad* es el evitar la necesidad de mejoras, por un lado, y por otra parte desarrollar software que sea fácil de mantener. La necesidad de mantenimiento es muy alta e impide los desarrollos nuevos. El porcentaje más alto del mantenimiento es para las mejoras propuestas por los usuarios y para mejorar la documentación —tareas que se pueden evitar o al menos reducir mediante una ingeniería de sistemas apropiada—.

Los *diagramas de estructura* son una herramienta importante para el diseñador de software. Muestran visualmente la relación entre los módulos del programa y los datos comunicados entre cada módulo. En general, se transmiten dos tipos de datos. Los *parámetros* son aquellos datos que el módulo necesita para el procesamiento. Por el contrario, la *información de control* ayuda a controlar el proceso, indicando la ocurrencia de errores o condiciones que afectan el proceso, tal como el indicador de fin de archivo.

Seis principios caracterizan a los buenos diseños de software: fragmentación descendente, *acoplamiento* holgado, agrupación funcional para la *cohesión*, extensión de control limitado, tamaño de adecuado de los módulos y módulos compartidos. Al seguir estos

principios se aumenta la probabilidad de alcanzar niveles aceptables de confiabilidad y facilidad de mantenimiento.

Las necesidades de mantenimiento y de aseguramiento de la calidad también se cumplen mejor cuando se usa cualesquiera de las tres herramientas de desarrollo estructurado y documentación: diagramas de flujo estructurado, diagramas de HIPO o de Warnier/Orr. Todos usan la notación gráfica para describir sistemas o módulos. Los *diagramas de flujo estructurado*, también llamados diagramas de *Nassi-Schneiderman*, definen los módulos de un sistema usando las tres estructuras básicas de proceso, decisión e iteración. Cada una se ensambla de forma descendente para especificar la lógica de un módulo o sistema.

HIPO (entrada-proceso-salida jerárquica) también es un diagrama gráfico del sistema y está formado por una *tabla visual de contenido* que describe el sistema en general, y un conjunto de diagramas funcionales. Cada diagrama muestra la entrada, salida, pasos del proceso y flujo de los datos. Los diagramas de HIPO son muy adecuados para documentar un sistema.

Los *diagramas de Warnier/Orr* muestran en forma explícita las relaciones jerárquicas entre los procesos y los subprocesos. Para desarrollar un diagrama de Warnier/Orr, el analista trabaja hacia atrás, empezando con la salida del sistema y definiendo el sistema cada vez con más detalle. Los requerimientos de datos se añaden para cada proceso. Estos diagramas sencillos y fáciles de entender son una forma excelente de mostrar las relaciones entre los procesos que integran un sistema.

El aseguramiento de la calidad también incluye pruebas para garantizar que el sistema se desempeña de forma adecuada y que cumple con sus requerimientos. Los casos especiales de prueba son la *validación*, *verificación* y *certificación*. El propósito de la prueba es hallar errores, no el demostrar lo correcto de un sistema. Los *casos de prueba*, que usan datos reales o artificiales, son procesados por el software y se reportan los errores.

Se usan dos estrategias de prueba, ninguna de las cuales es ideal o suficiente. En la *prueba de código*, el analista desarrolla casos de prueba para ejecutar cada instrucción o ruta dentro de un programa. En la *prueba de especificación*, el analista examina las especificaciones del programa y entonces escribe los datos de prueba para determinar cómo opera el programa bajo condiciones específicas.

Los dos niveles de prueba son las pruebas parciales y la prueba del sistema. En la primera, el analista, usando alguna de las estrategias anteriores, examina los programas que conforman un sistema. Por el contrario, la prueba de sistemas pretende hallar cualquier discrepancia entre el sistema y sus objetivos originales.

Existen seis pruebas especiales que son: la prueba de carga máxima, almacenamiento, tiempo de ejecución, recuperación, proce-

dimiento y de factores humanos. Cada una se centra en hallar los defectos de operación en el sistema para prevenir su falla.

Tanto los datos reales como los artificiales se usan para probar el sistema. Algunas organizaciones guardan los datos en *bibliotecas de prueba* para garantizar que todos los sistemas relacionados pueden procesar un conjunto común de datos de prueba cuidadosamente preparados. Además de la prueba inicial, las bibliotecas de prueba proporcionan datos confiables para probar el sistema cuando evolucione.

Las fallas en la prueba se muestran rápidamente cuando el sistema se implanta. La implantación del sistema se estudia en el capítulo siguiente.

PREGUNTAS DE REPASO

1. ¿Qué es la confiabilidad de un sistema? Comente los enfoques de confiabilidad de un sistema, señalando las ventajas y desventajas de cada uno. ¿Qué enfoque prefiere? ¿Por qué?
2. ¿En qué difieren los errores de hardware y software?
3. ¿En qué categorías se dividen las actividades de mantenimiento de un sistema? ¿Cuál es la frecuencia relativa de cada una?
4. ¿Qué niveles de modularidad caracterizan al software? ¿Cómo varían los distintos niveles?
5. ¿Qué son los diagramas de estructura? Comente sus componentes. ¿Qué papel juegan los diagramas de estructura en el diseño de software?
6. ¿Cuáles principios guían el diseño de software? Para cada uno, describa brevemente la intención del principio. ¿Qué resultados pueden ocurrir si no se siguen los principios?
7. ¿Qué es acoplamiento y cohesión? ¿En qué difieren y en qué se parecen? Dé ejemplos de cada uno, usando una aplicación representativa de sistema de información.
8. Comente el concepto de diagramas de flujo estructurado. ¿Cuáles son los elementos o componentes de los diagramas de flujo estructurado?
9. ¿Qué es HIPO? Comente el concepto y sus componentes. ¿En qué difiere HIPO de los diagramas de flujo estructurado? ¿Y de los diagramas de Warnier/Orr?
10. Comente las características de los diagramas de Warnier/Orr para el desarrollo de sistemas. ¿Cuáles son las ventajas de los diagramas como herramientas de diseño comparadas con otros métodos en este capítulo?
11. ¿Qué es el aseguramiento de la calidad en lo que respecta al software? ¿Por qué es un aspecto importante de la ingeniería de sistemas?
12. Comente los tipos más comunes de aseguramiento de la calidad. Dé ejemplos de cada tipo.
13. ¿Cuál es la finalidad de la prueba de software? Explique con ejemplos.
14. ¿En qué difieren la prueba de código y la prueba de especificación? Explique el propósito de cada una.
15. Comente las seis pruebas especiales de un sistema. Explique el propósito de cada una. Dé ejemplos para mostrar el propósito de la prueba.
16. ¿Cuáles son los niveles de prueba de un sistema? Identifíquelos y explique cada uno.
17. ¿Deben usarse datos reales o artificiales durante las pruebas de un sistema? Explique su razonamiento.

PROBLEMAS DE APLICACIÓN

1. Elabore un diagrama de estructura que describa la situación siguiente, en la que los pedidos de clientes se procesan en una organización.

Todos los pedidos se identifican mediante un número único de pedido. Todos los artículos vendidos por la compañía también tienen un número único de producto.

El procedimiento del sistema que se describe será automatizado aplicando buenas prácticas de diseño modular. Un operador de pedidos recibirá los pedidos por teléfono mientras trabaja en una terminal en línea. Se tomará la información por teléfono, y se introducirá al sistema por medio de una terminal.

Cuando se recibe un pedido de un cliente en la compañía, se asigna el número consecutivo de pedido para identificar el pedido. Además, se introduce el número de cuenta del cliente en el pedido, junto con el nombre y la dirección. Si la persona que hace el pedido ya es un cliente regular de la empresa, su nombre, dirección y dirección de envío tienen que estar en el archivo, identificados por el número de cuenta. De ser así, la información debe poder recuperarse cuando el cliente proporcione el número de cuenta. Sin embargo, si el cliente es nuevo, el sistema pedirá al operador de la terminal que introduzca el nombre del cliente, su dirección, dirección de envío e información bancaria y de crédito.

Todos los artículos de un pedido se introducen mediante el número del producto. Si el cliente le dice al operador el número del producto, éste lo introduce a la terminal, junto con la cantidad a ordenar. A su vez, la descripción y costo del producto se muestran en la terminal junto con una anotación de que los artículos están ahora "pedidos por el cliente". Cuando el cliente no conoce el número del producto, el operador introduce la clase de producto, dependiendo de la descripción que proporcione el cliente. En la pantalla de la terminal aparecen todos los artículos de esa clase. El operador y el cliente eligen conjuntamente el artículo correcto y se introduce su número de producto en el sistema, como se describió en el párrafo anterior.

Al final del pedido, el sistema muestra un listado de todos los artículos y cantidades solicitadas, los costos individuales y totales por cada artículo y el costo total global del pedido.

Al final del día, se imprime una copia de cada pedido y se la manda al cliente. Además, se prepara un reporte que resume todos los pedidos del día. El reporte incluye el número de cliente, nombre, dirección y dirección de envío, al igual que el monto total de cada pedido. Al preparar el diagrama de estructura:

- a. Muestre todos los módulos del sistema, identificando cada uno con un nombre descriptivo de su función. Use la notación estándar.
 - b. Muestre la secuencia de llamadas entre módulos.
 - c. Muestre los datos transferidos entre módulos.
 - d. Anote cualquier hipótesis que haga al preparar el diagrama.
 - e. Use un buen diseño modular.
2. El proceso de actualización de archivos secuenciales es común en todos los sistemas de información automatizados, independientemente del medio de almacenamiento usado para guardar transacciones y el archivo maestro. Si el archivo de transacciones se clasifica igual que el archivo maestro, los pasos son bien conocidos:
 - Preparar el archivo maestro para su uso.
 - Preparar el archivo de transacciones para procesamiento.
 - Leer un registro del archivo maestro.
 - Leer un registro del archivo de transacciones.

Si las llaves del archivo maestro y el archivo de transacciones concuerdan, el registro maestro se modifica y se escribe de nuevo en la memoria y los nuevos registros maestro y de transacción se leen en la memoria.

Si la llave del archivo de transacciones es menor que la llave del archivo maestro, se crea un registro y se escribe en el archivo maestro y se lee el siguiente registro del archivo de transacciones.

Si la llave del archivo de transacciones es mayor que la llave del registro del archivo maestro, el registro maestro se escribe en el nuevo archivo, se lee el nuevo registro maestro y el proceso se repite.

El proceso anterior se repite hasta que no haya más transacciones o registros en el archivo maestro. Entonces se cierran los archivos.

- a. Desarrolle un diagrama de flujo estructurado para el procedimiento de proceso de datos descrito.
- b. Desarrolle un diagrama de Warnier/Orr para el procedimiento.
- c. ¿Cómo cambiarán los diagramas si se supone que
 - (1) ¿Hay más de una transacción para aplicarla a un registro específico del archivo maestro?
 - (2) ¿El archivo maestro se organiza con una organización de acceso directo?

3. Un patrón común de diseño que se usa cuando los individuos deben interactuar con un sistema en línea mediante pantallas, usa una serie de menús. Cada menú informa al usuario de las opciones que puedan ejecutarse en ese punto. En un sistema, el usuario entra introduciendo la palabra clave STARTUP (cualquier otra palabra clave es un error) y el sistema responde mostrando el menú principal del sistema. Las opciones del menú principal son la de añadir nuevos registros al archivo, editar los registros existentes, generar reportes, reindexar el archivo principal o salir del sistema.

Si el usuario elige la opción de reporte, se presenta otro menú a partir del cual el usuario puede elegir uno de cuatro reportes. Después de que el reporte ha sido seleccionado, se muestra otra pantalla preguntando al usuario si el reporte debe estar en orden alfabético, numérico o cronológico.

El usuario puede salir del sistema desde cualquier menú o puede regresar al siguiente menú de nivel superior. La salida se hace oprimiendo x o X. El movimiento de un menú a otro se hace oprimiendo m o M. Todas las opciones se eligen mediante un número (1, 2, 3, 4, etc.)

- a. Desarrolle una tabla visual de contenido de HIPO para este diseño.
 - b. Describa el proceso por medio de un diagrama de Warnier/Orr.
 - c. Muestre la entrada al sistema y el proceso de selección de menú en forma de diagrama de flujo estructurado.
4. Los documentos que producen las empresas para enviar mercancías a los clientes a menudo reciben el nombre de *documentos de envío*. Análogamente, el sistema que produce los documentos se conoce como un *sistema de documentos de envío*. Un sistema particular utiliza un archivo de envíos para producir los documentos apropiados para el transporte de mercancía a los clientes, los cuales se clasifican con un código único de dos caracteres que describe el tipo de cliente y forma de envío a usar. Si el código de envío es BK, DV o CN, el programa añade un nuevo registro de envío al archivo de transacciones de envíos, archivo que se crea para su uso posterior por otro módulo del sistema. Si el código de envío es DA y el destino es un código de planta de producción DA, o si el código de envío es HH y el destino es una planta de producción con código DA, también se añade un registro de transacción al archivo de transacciones de envíos. El otro caso donde el sistema debe crear un registro de transacción de envío es si el código de pedido es QZ5.

Para estas situaciones y los demás códigos, el sistema produce una factura (además de la creación de los registros de transacción de envío descritos). Las existencias en el archivo de inventario de mercancías se reduce en el número de unidades facturadas cuando se prepara el documento de envío. Si el número de artículo en el documento de envío no coincide con ningún número de artículo en el archivo de inventario, el archivo no se altera y se produce un mensaje de error. Sin embargo, el documento de envío se sigue preparando como ya se describió.

- a. Desarrolle un diagrama de HIPO mostrando la entrada, proceso y salida para el sistema de envío tal y como se ha descrito.
 - b. Muestre la lógica del diseño usando diagramas de Warnier/Orr.
 - c. Compare los diagramas desarrollados en *a* y *b*. ¿Qué ventajas ofrece un método sobre el otro? ¿Cuáles son las desventajas de cada método?
5. Un analista diseñó un sistema de informes a la dirección que proporciona un reporte de una sola página donde resume las ventas y gastos del día. Las actividades de cada uno de los tres departamentos principales se reporta por separado, así como el total de los otros departamentos juntos.

Además, el sistema reporta las ventas acumuladas durante ese año para cada departamento y en total. Lo hace recuperando las ventas del día anterior y añadiendo las ventas del día de hoy a ese total. Después se graba el nuevo total junto con la fecha de hoy.

El sistema se diseñó de tal forma que haya un registro grande de datos por cada día. En un año normal, el archivo contiene 365 días. Cada número de registro es el día numérico del año. Por ejemplo, el dato del primero de enero se guarda en el registro 1, el 31 de enero en el registro 31, el primero de febrero en el registro 32 y así sucesivamente. Cuando el usuario entra al sistema, hay que introducir la fecha actual. A su vez, el sistema traduce esto a un valor numérico.

Si la fecha de hoy es el 31 de enero, el sistema recuperará las ventas acumuladas hasta la fecha del registro 30 (para el 30 de enero) y se añaden las ventas del día 31 para formar un nuevo total acumulado hasta el 31 de enero. Ese total se guardará como parte del registro 31 pero el registro 30 no se modifica. Al final del año, el sistema se recicla. En otras palabras, después del 31 de diciembre, el sistema empieza a reusar el archivo comenzando con el registro 1 (para el primero de enero).

Desarrolle casos de prueba para determinar si el sistema recupera los datos del día anterior en forma adecuada. Especifique todas las combinaciones de datos necesarios para garantizar que la recuperación del día anterior trabaja adecuadamente. No hay que olvidar el año bisiesto, en el que hay 29 días en febrero y 366 días en el año.

6. Escriba el conjunto de los datos de prueba que crea necesarios para probar en forma adecuada el procedimiento de actualización de archivos descritos en la pregunta 4.

BIBLIOGRAFÍA

- BOEHM, B.W.: "Improving Software Productivity", *Computer*, 20, 9, septiembre de 1987, pp. 4357.
- BOEHM, B.W.: "Software Engineering"; *IEEE Transactions on Computers*, 25, 12, diciembre de 1976, pp. 1226-1241.
- BROOKS, F.P. JR.: *The Mythical Man Month*, Reading, MA: Addison Wesley, 1978.

- HECKEL, P.: *The Elements of Friendly Software Design*, NY: Warner Books, 1984.
- HIGGINS, D.A.: *Program Design and Construction*, Englewood Cliffs, NJ: Prentice-Hall, 1979.
- JACKSON, M.A.: *Principles of Program Design*, NY: Academic Press, 1975.
- LIENTZ, B.P. y E.B. SWANSON: *Software Maintenance Management*, Reading, MA: Addison Wesley, 1980.
- METZGER, P.W.: *Managing a Programming Project*, 2a. edición, Englewood Cliffs, NJ: Prentice-Hall, 1981.
- MYERS, G.J.: *The Art of Software Testing*, NY: Wiley-Interscience, 1979.
- MYERS, G.J.: *Software Reliability*, NY: John Wiley and Sons, 1976.
- NASSI, I. y B. SCHNEIDERMAN: "Flowchart Techniques for Structured Programming", *SIGPLAN Notices*, 8, 8, agosto de 1973, pp. 12-26.
- PARNAS, D.L., P.C. CLEMENTS y D.M. WEISS: "The Modular Structure of Complex Systems", *IEEE Transactions on Software Engineering*, 34, 3, marzo de 1985, pp. 259-266.
- PETERS, L.J.: *Software Design: Methods and Techniques*, NY: Yourdon Press, 1981.
- STEVENS, W.P., G.L. MEYERS y L.L. CONSTANTINE: "Structured Design", *IBM Systems Journal*, 13,2, mayo de 1974, pp. 115-139.
- WARNIER, J.D.: *Logical Construction of Programs*, NY: Van Nostrand Reinhold Company, 1974.
- WASSERMAN, A.I. y STEVEN GUTZ: "The Future of Programming", *Communications of the ACM*, 25, 3, marzo de 1982, pp. 196-206.
- YOURDON, E. y L.L. CONSTANTINE: *Structured Design*, Englewood Cliffs, NJ: Prentice-Hall, 1979.

15. Administración del proceso de implantación del sistema

GUÍA DE ESTUDIO

Usted entenderá bien cómo administrar el proceso de implantación de los sistemas de información cuando pueda responder a estas preguntas:

- ¿Qué aspectos deben considerarse en la fase de capacitación de los usuarios y los operadores?
- ¿Cómo varían los métodos de capacitación entre las organizaciones?
- ¿Qué aspectos de la conversión de sistemas son los más difíciles de administrar?
- ¿Existen estrategias de conversión variables?
- ¿Cómo aseguran los analistas de sistemas que la conversión se lleva a cabo de forma adecuada y precisa?
- ¿Porqué es necesaria la revisión después de la implantación cuando los sistemas se instalan?
- ¿Cuáles son los métodos que se usan para revisar los sistemas después de la implantación?



OBJETIVOS DEL CAPÍTULO

- Evaluar un programa de capacitación para determinar si es completo y si tiene posibilidad de éxito.
- Preparar un plan de conversión para obtener un nuevo sistema de información.
- Diseñar un procedimiento para la conversión de los datos y archivos ya existentes para utilizarlos en el nuevo sistema.
- Delinear un plan de revisión después de la implantación.

PALABRAS CLAVE

Bitácora de eventos
Conversión
Conversión directa
Enfoque piloto
Etapas

Evaluación del impacto
Gerente de conversión
Procedimientos de ejecución
Revisión después de la implantación
Sistemas paralelos

Vacaciones en casa

Linda Gianni ha estado pensando cómo tomará su personal la implantación del nuevo sistema en su oficina. Todos parecen estar entusiasmados con la computarización, aun cuando ninguno de los miembros del personal, excepto ella, han tenido experiencia directa con las computadoras. Después de todo, en la mayoría de los casos, ellos están familiarizados con estas máquinas, ya que tienen cónyuges o hijos que usan computadoras en el trabajo o en la escuela. Al existir computadoras virtualmente en toda empresa, banco, tienda y terminal aérea, los temores y las preocupaciones que hace algunos años la gente tenía con respecto a la computarización han desaparecido en gran medida. El personal reconoce que las computadoras son parte integral de las empresas, así que están a favor del cambio. Se dan cuenta que les ahorrarán tiempo y mejorarán el servicio.

El departamento de Linda es el responsable de mantener registros exactos de los 5000 vehículos en renta de la compañía; el nuevo sistema promete hacer más fácil el trabajo para ella y sus compañeros de trabajo. Cada registro individual mostrará la fecha y el precio de compra del vehículo, la historia del mantenimiento y los términos del convenio de renta. Además, el sistema eliminará la necesidad de procesar manualmente los estados de cuenta mensuales que se envían a los arrendatarios, procedimiento que el personal desea automatizar. Como gerente, Linda se preocupa por las necesidades y deseos de sus compañeros de trabajo, así como de las necesidades y deseos de la empresa. Por fortuna, el nuevo sistema tiene el potencial para satisfacer a todos.

Linda revisa de nuevo la carta del proveedor que muestra los planes de capacitación para su departamento.

El programa de capacitación para el nuevo sistema será de cuatro días:

- | | |
|--------------|---|
| Primer día: | conceptos de computación, incluyendo una introducción al hardware, un panorama de las funciones que realiza el software y procedimientos generales de procesamiento de datos. |
| Segundo día: | experiencia directa con los menús, funciones y características del sistema. |
| Tercer día: | creación de archivos, introducción de datos, preparación de reportes e impresión de salidas. |
| Cuarto día: | práctica supervisada de uso del sistema. |

Durante las sesiones de capacitación, el personal comenzará a construir archivos para su propio uso e introducirá los datos reales que ellos necesitan para procesar las cuentas. Esto asegurará que la capacitación sea realista, efectiva y también útil.

En vista de que se acerca el periodo de vacaciones, hemos tratado de preparar una programación que no interfiera con los planes vacacionales de los empleados.

Como resultado, hemos programado la capacitación para impartirse del 1 al 20 de diciembre, periodo de tres semanas que consideramos adecuado para las veinte personas que recibirán la capacitación.

Al final de la capacitación, daremos un receso hasta el 5 de enero, fecha en la cual adoptaremos el nuevo sistema. Todo el equipo se instalará y probará antes del primero de diciembre. Entre el 30 de diciembre y el 5 de enero probará el hardware, se harán ajustes al software o se corregirá cualquier dificultad que surgiera durante la capacitación. Ninguna de estas actividades interferirá con las operaciones regulares en su oficina.

Linda dejó la carta y empezó a pensar en las implicaciones de la programación propuesta. ¿Es una buena idea el receso de vacaciones?, se dijo a sí misma. Todos quieren el sistema en su lugar, pero tal vez la capacitación y la conversión debieran retrasarse hasta después de las vacaciones para que haya continuidad. Por supuesto, el proveedor probablemente no tenga ese espacio de tiempo disponible para la capacitación, dado un aviso tan precipitado.

También existe el problema de asegurarse que los registros se introduzcan de manera apropiada. Al tener que introducir los registros de 5000 vehículos por medio de las terminales, habrá muchas posibilidades de cometer errores u omisiones, recordó. ¿Será una buena decisión que el personal "practique" construyendo archivos con los datos reales? ¿No existe un gran peligro de equivocarse?

Al examinar la situación, Linda se preguntó: ¿cuál es el mejor método para dirigir la implantación? La programación propuesta, ¿dará por resultado una capacitación y conversión de la mejor calidad posible?

La implantación incluye todas aquellas actividades que tienen lugar para convertir del sistema anterior al nuevo. El nuevo sistema puede ser totalmente nuevo y reemplazar al sistema que hay, ya sea manual o automatizado; o bien puede ser una modificación importante de un

	CAPACITACIÓN DEL PERSONAL DE SISTEMAS	CAPACITACIÓN DEL PERSONAL USUARIO
Participantes	Personal de preparación de datos Personal de captura de datos Operadores de computadora	Todos los usuarios directos o indirectos
Temas principales	Uso de equipo Identificación de los problemas del equipo Procedimientos de ejecución de la computadora Programación de trabajos de la computadora Programación de actividades Mantenimiento del sistema	Uso del equipo (en caso necesario) Identificación de los problemas del equipo (en caso necesario) Familiarización con la aplicación Captura y codificación de los datos Manejo de datos: añadir registros eliminar registros editar registros Recuperación de información Uso de la información
TIPOS DE CAPACITACIÓN		
	Proporcionada por el vendedor:	Uso del equipo Operación directa Características y controles del software
	"En casa":	Igual que arriba, pero se puede adaptar a las necesidades del usuario. Se puede usar material audiovisual, manuales de capacitación especial o software especial.

FIGURA 15.1
Actividades de capacitación de sistemas.

sistema existente. En cualquier caso, la adecuada implantación es esencial para lograr un sistema confiable y que cumpla con las necesidades de la organización. Una implantación exitosa no garantiza el mejoramiento de la organización que use el nuevo sistema (eso es una cuestión de diseño), pero su instalación inadecuada lo impedirá.

Como lo muestra este capítulo, aun los mejores sistemas pueden debilitarse si los analistas que conducen la implantación no atienden cada detalle importante. Ésta es un área en la que los nuevos analistas de sistemas deben concentrar gran parte de su atención. Los *buenos analistas* pueden hacer una gran diferencia.

Este capítulo estudia los tres aspectos de la implantación: capacitación de personal, procedimientos de conversión y la revisión después de la implantación. En cada área, se estudian los elementos particula-

res de ese aspecto, así como los métodos de manejo de cada situación de forma eficiente y efectiva.

CAPACITACIÓN

Aun los sistemas técnicamente elegantes y bien diseñados pueden tener éxito o fracasar debido a la forma en que se operan y usan. Por lo tanto, la calidad de la capacitación recibida por el personal relacionado con el sistema ayuda u obstruye, y puede llegar a impedir, la implantación exitosa de un sistema de información. Aquellos que estén asociados con el sistema o afectados por el mismo deben conocer con detalle cuáles serán sus papeles, cómo pueden usar el sistema y qué hará o no hará el sistema. Tanto los operadores como los usuarios del sistema necesitan capacitación (Fig. 15.1).

Capacitación de operadores de sistemas

Muchos sistemas dependen del personal del centro de cómputo, el cual es responsable de mantener al equipo funcionando, así como de proporcionar el servicio de apoyo necesario. Su capacitación debe asegurar que puedan manejar todas las operaciones posibles, tanto rutinarias como extraordinarias. La capacitación del operador también debe contemplar al personal de captura de datos.

Si el sistema necesita la instalación de nuevo equipo, por ejemplo, un nuevo sistema de computación, terminales especiales, o equipo distinto para la captura de datos, la capacitación de los operadores debe incluir aspectos tan básicos como saber prender el equipo, usarlo, apagarlo, y también un conocimiento de lo que es su operación y uso normales. También debe capacitarse a los operadores acerca de los desperfectos más comunes, cómo reconocerlos y qué pasos llevar a cabo cuando ocurran. Como parte de su capacitación, se debe dar a los operadores una lista de formas de resolver los problemas y que identifique los posibles problemas y solución, así como los nombres y números telefónicos de las personas a quién buscar cuando surjan problemas inesperados.

La capacitación también necesita la familiarización con los *procedimientos de ejecución*, lo cual implica trabajar a través de la sucesión de actividades necesarias para usar un nuevo sistema. Estos procedimientos permiten a los operadores de la computadora familiarizarse con las acciones que deben realizar (como montar discos o cintas magnéticas, copiar archivos, cambiar las formas de impresión o hacer funcionar sistemas de información) y saber cuándo deben ocurrir dichas acciones. Además, sabrán cuánto tiempo le llevará a las aplicaciones correr bajo condiciones normales. Esta información es importante, tanto para que los usuarios puedan planear su trabajo como para identificar los sistemas que corran más rápido o más lento de lo esperado —una señal que usualmente indica problemas con la corrida—.